

分类号_____

学校代码 10487

学号 M201876074

密级

华中科技大学

硕士学位论文

基于 SpringBoot 的医院门诊管理信息 系统的设计与实现

学位申请人：胡小勇

学科专业：软件工程

指导教师：区士颀

答辩日期：2021.5

**A Thesis Submitted in Partial Fulfillment of the Requirements
for the Degree for the Master of Engineering**

**Design and Implementation of Hospital
Outpatient Management Information System
Based on SpringBoot**

Candidate : Xiaoyong Hu

Major : Software Engineering

Supervisor : Shiqi Ou

Huazhong University of Science & Technology

Wuhan 430074, P.R.China

May, 2021

摘要

随着新时代的来临，各行各业都悄然的发生了巨大的变化，将计算机技术应用到日常生活和社会生产中已成为常态。医疗领域正朝着信息化、智能化的方向迈进，医院门诊管理信息系统是医院信息系统的一个分支，作为患者看病就诊的第一站，也作为患者接触知晓医院的首要门户，门诊的服务质量直接与患者对医疗机构的第一印象相关联。就目前来看，部分中小型医院未完全进行信息化改造，只简单的实现了医院管理系统的初步使用。为了给患者提供更好的就医服务，简化就医流程，提高信息资源的共享度，减轻工作人员的工作强度和运维成本，开发一款面向患者的门诊信息管理系统来满足诸多中小型医院的需求，实现高效管理其业务流程，对于医院来说是十分有必要的。

系统采用当前流行的B/S架构模式，使用前后端分离的开发方式，设计并实现基于SpringBoot的医院门诊管理信息系统。本文首先了解医院信息系统的国内外发展现状，明确了系统研发的目的及意义。在需求分析中，通过现场实践走访，分析了该系统的功能性需求，划分了用户信息管理、医生排班管理、预约挂号管理、门诊医生工作台、门诊收费管理及门诊药房管理六个主要功能模块。在设计阶段，从架构选型及系统体系设计阐述系统的总体设计，对各个功能模块采用流程图及时序图进行表述，最后采用E-R图及表结构的形式对数据库展开设计。在实现及测试阶段，以Web界面的形式展示了各功能模块的实现效果，且对核心功能给出相应的代码实现逻辑，最后使用功能测试及性能测试检验了系统的功能实现情况与性能结果，成功达到需求分析设定的目标，功能完善，性能符合预期。

本系统囊括中小型门诊医院的业务流程所需功能，符合医院的流程信息化、办公高效化、信息共享化、病历无纸化标准，将显著提高医院产生的经济效益及社会效益。

关键词： 门诊信息管理 信息共享 B/S 架构 SpringBoot

ABSTRACT

With the advent of the new era, great changes have taken place in all walks of life. It has become normal to apply computer technology to daily life and social production. The medical field is moving in the direction of informatization and intelligence. The hospital outpatient management information system is a branch of the hospital information system. It serves as the first stop for patients to see a doctor and the primary portal for patients to get in touch with the hospital and the quality of outpatient services. It is directly related to the patient's first impression of the medical institution. From the current point of view, some small and medium-sized hospitals have not completely carried out information transformation, but simply realized the initial use of the hospital management system. In order to provide patients with better medical services, simplify medical procedures, improve the sharing of information resources, and reduce the work intensity and operation and maintenance costs of staff, a patient-oriented outpatient information management system has been developed to meet the needs of many small and medium-sized hospitals. It is very necessary for hospitals to realize efficient management of its business processes.

The system adopts the current popular B/S architecture model, and uses the front-end and back-end development method to design and implement a hospital outpatient management information system based on SpringBoot. This article first understands the domestic and foreign development status of hospital information system, and defines the purpose and significance of the system research and development. In the demand analysis, through on-site practice, the functional requirements of the system are analyzed, which are divided into six main functional modules: user information management, doctor scheduling management, appointment registration management, outpatient doctor workbench, outpatient charge management and outpatient pharmacy management. In the design stage, the overall design of the system is elaborated from the architecture selection and system design, and the flow chart and sequence chart are used to describe each functional module. Finally, the database is designed in the form of E-R

chart and table structure. In the implementation and testing stage, the implementation effect of each functional module is displayed in the form of a web interface, and the corresponding code implementation logic is given to the core function. Finally, the functional implementation and performance results of the system are verified by functional testing and performance testing. Successfully reached the target set by the demand analysis, with complete functions and performance in line with expectations.

This system includes the functions required by the business process of small and medium-sized outpatient hospitals, and meets the hospital's process informatization, office efficiency, information sharing, and paperless medical records standards. It will significantly improve the economic and social benefits of the hospital.

Key words: Outpatient information management Information sharing

B/S architecture SpringBoot

目 录

摘 要.....	I
Abstract.....	II
1 绪论	
1.1 研究背景与意义.....	(1)
1.2 国内外研究概况.....	(2)
1.3 研究内容.....	(3)
2 关键技术研究	
2.1 Spring 容器框架.....	(5)
2.2 Mybatis 持久层框架.....	(6)
2.3 Spring Boot 集成框架.....	(7)
2.4 Shiro 安全框架.....	(7)
2.5 Kafka 消息队列.....	(8)
2.6 本章小结.....	(9)
3 系统需求分析	
3.1 系统需求概述.....	(10)
3.2 功能需求分析.....	(11)
3.3 系统用户及用例分析.....	(12)
3.4 非功能需求分析.....	(16)
3.5 本章小结.....	(17)

4 系统设计

4.1 系统总体设计.....	(19)
4.2 系统功能模块设计.....	(22)
4.3 系统数据库设计.....	(38)
4.4 本章小结.....	(43)

5 系统实现与测试

5.1 系统开发环境.....	(44)
5.2 系统实现.....	(44)
5.3 系统测试.....	(67)
5.4 本章小结.....	(71)

6 总结与展望

6.1 全文总结.....	(72)
6.2 展望.....	(72)

致 谢.....	(74)
----------	------

参考文献.....	(75)
-----------	------

1 绪论

1.1 研究背景与意义

随着计算机时代的到来,各行各业都悄然的发生了巨大的变化,尤其对人们日常生活息息相关的衣食住行相关行业产生了莫大的影响,在互联网的大环境驱动下,医疗行业也在稳步的发展。与此同时在新政策的领导下,我国经济迅猛发展,人民生活水平随之提高,健康一词逐渐步入人心,对医院的服务水平要求也越发严格。

在发展的过程中,国家的总体发展方针为优先发展城镇,再带动乡村地区的发展,所以在城镇化建设过程中,八成以上的医疗物质分布在城市大型医院中,在不发达的地区医疗物质及水平相对短缺。另外面临缺乏资金、检测设备、计算机专业人才等问题,导致医院的信息管理只停留在半信息化的阶段上,患者到这些中小型医院看诊经常面临“就医难”的困扰^[1]。其次多数医院部门之间的数据共享程度不高,存在数据“孤岛”问题,患者做检查的结果图片无法得到共享,多次做检查浪费财力和时间,同时也增加了医技工作人员的负担,如果共用一套系统,病人的检查结果医生可以随时查看,这样可以很好的重复利用这些资源。

再此背景下,本文提出开发一个医院门诊信息管理系统,该系统包含用户信息管理、医生排班管理、预约挂号管理、门诊医生工作台、门诊收费管理、门诊药房管理六大部分。门诊医院管理系统的使用带来的好处有:

首先,从医院的角度上来说,如若医疗管理系统能在医疗信息领域广泛推广使用,将为医院的现代化、科学化管理打下技术基础,跟上时代信息化步伐。同时通过简化操作流程,减少医务人员的冗余步骤,提高其工作效率,间接的创造经济价值及社会价值,最终提高医院的服务水平及影响力。门诊系统是整个医院管理系统的一部分,通过系统将各个部门的数据信息进行共享,为管理者提供动态数据支撑便于其进行分析,及时调整各部门的人员工作安排,做到科学的门诊动态决策管理^[2]。

其次,从患者的角度上来说,通过此系统,患者可以进行自助台预约挂号,全面了解坐诊医生信息的同时解决了挂号时间长的问题;通过数据平台的数据流通共享,将检查结果通过平台传递,避免了重复执行检查的操作;随着智能手机的普及,

患者挂完号以后给患者来一个就医短信提醒，有助于减少患者因忘记预约时间而错过最佳就诊时机。

最后，从医生的角度上讲，减少了对患者疾病信息进行重复填写诊断记录的工作，把更多的精力放在诊断过程中，这样就能明显提高就诊率，使患者得以早日康复；电子病历平台可以方便医生查看以前患者的历史就诊记录，以便后序对患者进行精确诊断；利用云存储平台对检查结果的图像进行存储，提高图像存储的安全性及稳定性^[3,4]；通过将各部门的信息进行共享，医生可及时看到患者的检查、校验结果等报告；电子处方模板及自动检测处方药品数量等有助于减少因为药物产生的医疗事故。

1.2 国内外研究概况

早在二十世纪中期，西方发达国家就已经意识到了医院信息系统对于患者、医院和医疗行业的重要性^[5]。并着手开始研究相关的概念和技术，因此其能够较早地应用医院信息系统到实际的医疗救治活动中。在医院信息系统发展初期，以美国为首的发达国家，为了使医院财务的管理更具系统化，最先将财务系统引入到医院中^[6]。首先，财务系统与专业的医疗系统相对独立，财务有自己的一套管理、记账规范，与医疗专业知识重叠较少，引入进来不会造成医疗诊治流程的巨大变动。其次，财务系统对于各行各业来说是必不可少的，财务也是任何一个机构非常重要的部门，通过电子财务系统，能够提高财务的工作效率和质量，减少出现的错误。财务系统在医院中发挥的作用显著，也带动了医院信息系统的发展。伴随着社会发展及科技水平的进步，医院的规模也在逐渐扩大，部门慢慢增多，患者治疗与日俱增，对系统功能的多样性也提出了更高的需求。于是医院信息系统的功能也越来越多，越来越综合，大型的信息系统慢慢出现。据调查，二十世纪七十年代到九十年代，美国诸多大型医院逐步发展应用了完整的医院信息系统，四分之三的酒店有财务系统，一半以上的医院支持门诊挂号电子登记^[5]。可以看到，医院信息系统已经不仅仅局限于财务、行政管理，而是深入到疾病诊疗的过程当中，利用信息技术助力医生诊断疾病。同时，技术的发展也支持更加复杂的数据处理，更大的数据存储，更高清的医院影像。在这一时期，欧洲发达国家也在开展信息系统的研发与应用工作，虽然较美国发展晚一些，但是有技术支撑和美国的经验借鉴，因此发展迅速，大型医

院信息系统也很快得到应用。

进入二十世纪后，医院信息系统朝着医疗电子化、共享化、智能化发展。依然以美国为主导，早在世纪初，2004 年美国总统布什在报告中重申了 HIS 的建设方向，提出要加强医院信息系统建设，着重建立全民电子健康档案^[7]。这样做的好处有很多，一是就医无纸化，降低成本。二是电子档案可以长期保存，有助于跟踪患者就诊历史，在患者就医时，方便根据疾病史，让医生能做出更科学的判断；三是便于传输和共享，让医院之间能共享患者的就诊信息。综合来看，医疗档案电子化，有助于提高医生诊疗效率，减少医疗事故，促进患者早日康复，同时显著降低医院的运维成本，从而促进整个医疗行业的发展^[8]。

在国内，受限于资金、技术水平、医疗设备等条件的限制，我国医疗信息建设起步晚始于 20 世纪 70 年代初，在此起步阶段主要以单方面的医学研究为主^[9]，只有少数几家大型综合医院拥有这样的条件，主要的设备为 IBM 的 M340 小型机。步入 80 年代初期，1981 年成立的中国电子医药信息学会为我国 HIS 的发展提供了广大的交流平台，极大的促进了 HIS 系统的发展，是中国 HIS 发展历程上的一个里程碑^[10]。80 年代中下旬，伴随着各种网络操作系统的出现，如 Unix，部分医院利用其构建自己的局域网络信息并开发出部门管理系统，主要用于药品、住院等部门的管理。进入 90 年代，我国 HIS 系统进入快步发展阶段，软件开发规范逐步形成，有技术力量的医院及公司根据自定义需求开发完整的医院管理系统，极大地简化了办公流程，间接提升了医院工作人员的工作效率。进入 21 世纪新时代，在国家医疗政策的全面推动下，医院进行医疗体制改革优化，将计算机先进技术引入实现医院办公系统的信息化、科学化管理，降低医院运维成本，为医院带来经济及社会效益，同时也为患者提供便利、科学、全面的服务^[11,12]。HIS 系统也在不断变化的需求中发展和完善，力求为人民提供更便捷、科学、优质的服务。

1.3 研究内容

本文通过深入了解项目的背景及精确的对需求进行分析，结合医院的诊断流程，设计并实现一款解决患者“看病难”问题的医院门诊管理系统，从本质上解决门诊医院存在的“两长一短一低”的问题，即挂号时间长、就诊周期长、看诊时间短及数据共享程度低的困扰。从而转变医院信息的管理方式，缩短各个科室的业务处理

时间，从而为患者提供更优质的服务。

本人参与了项目开发的各个阶段，负责开发用户信息管理、医生排班管理、预约挂号管理、门诊医生工作台、门诊收费管理、门诊药房管理六大模块的详细设计与具体实现。本人的工作主要工作有：

第一，在需求明确后，采用主流的 B/S 架构进行设计，利用 Java 轻量级框架 SpringBoot 进行开发，结合实际业务，采用关系型 MySQL 及缓存 Redis 来实现数据存储，使用 Kafka 消息队列的异步机制将额外业务分离，实现业务解耦。

第二，在对患者的检查、校验结果进行存储时，由于医学图像需要的存储空间比较大，安全性及稳定性要求比较高。云存储服务由其可扩展、成本低、多租户等特点被大家广泛使用，本文综合考虑使用阿里云的 OSS 对象存储服务对医学图像进行存储。

第三，功能模块设计阶段，结合 UML 工具，对每个核心模块的功能使用流程图或时序图进行生动的展示，接着开始每个模块的编码过程，以页面实现效果图及代码的形式阐述部分核心功能。最后对系统整体进行功能及非功能性测试，以检测其是否达到预期的需求。

2 关键技术研究

本章针对医院门诊管理系统开发过程中用到的关键技术进行介绍，主要技术包括 Spring 容器框架、Mybatis 数据交互持久层框架、SpringBoot 集成框架、安全框架 Shiro，使用 Kafka 消息队列来对消息进行延迟发送。使用 Mysql 进行关系型数据存储，对热点数据采用 Redis 缓存进行存储，借助 Nginx 实现服务负载均衡^[13]，以及 Maven、Git、Jenkins 等版本控制与项目管理工具。

2.1 Spring 容器框架

Spring 是非常优秀的框架，由许多模块组成，Spring 基于核心容器进行构造，定义了创建与管理 Bean 的方法，Spring 的模块如下图 2-1 所示

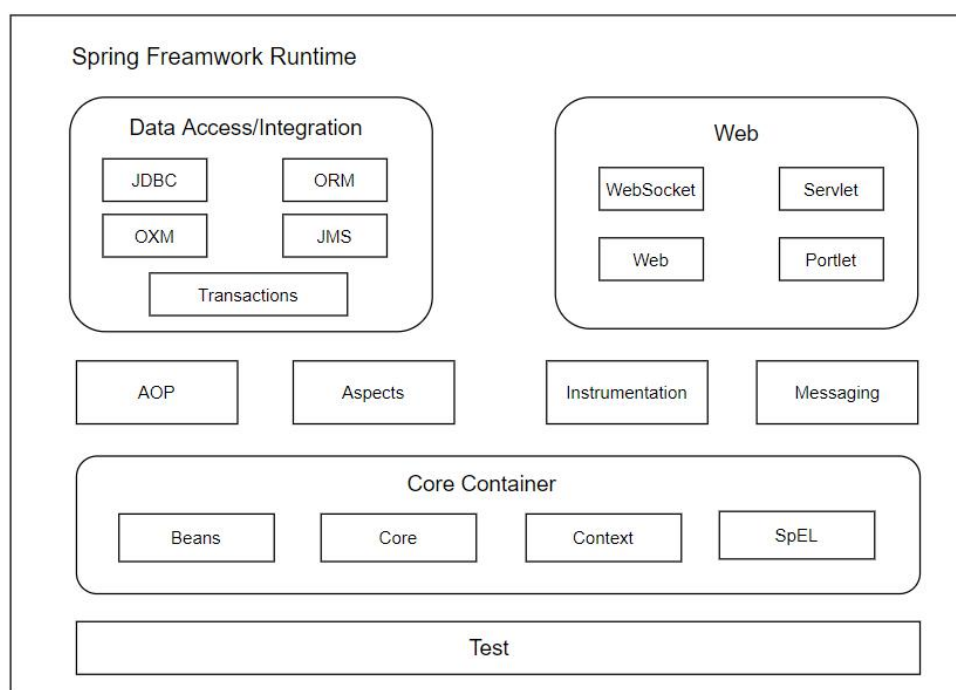


图 2-1 Spring 框架的构成

核心容器(Core Container)是提供核心特性的模块，利用其核心组件 Bean 工厂对应用所涉及的单实例 Bean 对象执行实例化操作，同时对其依赖的关系对象进行注入配置，然后交给 IOC 容器进行统一协调管理，其依赖的是 IOC 控制反转和 DI 依赖注入特性^[14]。采用这种第三方进行管理的方式，有助于降低对象之间的耦合度、

使对象管理更高效化。

Spring 通过面向切面（AOP）框架使其具备面向方面编程的特性，AOP 框架底层通过动态代理的模式来进行实现，在编译运行过程中，通过在业务逻辑的前后动态的织入一些通用逻辑如事务、日志逻辑等，即可将这些功能切入到业务逻辑中去，这样就将业务逻辑与通用功能进行了分离，降低了功能间的耦合度，进而保证了业务的原子性操作^[15-17]。

Spring 通过集成 ORM 框架进行整合，提供了对流行的数据交互框架的支持，例如 Hibernate、Mybatis 等 ORM 框架，通过整合这些框架使得开发人员能迅速的定位及解决在使用数据库时遇到的问题。ORM 框架充当将实体对象数据与数据库表中的关系型数据进行互相转换的角色，把实体对象模型映射到数据库表的关系模型中去，最终只用操作实体对象的属性即可完成数据的读取或存储。

Spring 团队最初没有打算构建 Web MVC 框架，但是其开发人员在开发过程中发现先有的 Struts 框架用来处理展示层与业务请求处理层及业务请求处理层与模型之间的分离程度不够，所以随之开发了 Spring MVC^[18]。

2.2 Mybatis 持久层框架

Mybatis 是一个持久层框架，其对数据库的底层操作细节进行了封装，使上层开发流程变得更加简单明了。其通过 XML 配置文件将数据访问（Data Object Access）层的接口与实体类的映射 Mapper 文件进行一一关联，映射文件 Mapper 里面配置了操作每个实体对应数据库表的 SQL 语句的映射，每个语句对应接口的一个具体方法^[19]。与传统的 ORM 框架相比，Mybatis 具有如下优点：

（1）传统的 ORM 框架是将实体类完全映射到数据库表中，然后调用接口完成对数据库表中数据的操作，而 Mybatis 则是在映射文件中将对应接口映射为对应的 SQL 语句，最后在数据库执行 SQL 语句即可。

（2）传统的框架推荐让其自动生成 SQL 语句，这样在数据关系比较复杂时候，生成的 SQL 语句也极其复杂，导致生成 SQL 语句往往不能按照预期使用正确的索引结构，最终出现数据查询慢等问题。而 Mybatis 推荐自己控制操作数据库表的 SQL 语句，这样就可以充分利用数据库的许多关键特性，可以更加灵活的预数据库进行交互。

(3) Mybatis 提供两种不同等级的缓存,最常用的为会话等级 SQLSession 的缓存,通过将该会话首次执行 SQL 语句的结果进行缓存,以便下次执行相同的语句时从缓存中读取,而非数据库。其次则是应用等级的缓存,该框架提供了集成第三方缓存的连接池^[20]。比如 EHCache、OSCache 缓存等,通过集成第三方缓存,可以减轻数据库的压力,提高数据库性能。

2.3 Spring Boot 集成框架

Spring Boot 是 Pivotal 团队 2013 年开发,第一个版本于 2014 年发行,该框架继承了 Spring 中众多的特征,是第一个全新的轻量级开发框架,它的出现简化了项目开发过程中各种繁杂的配置文件的编写以及解决了项目依赖包之间的冲突,使开发者更加专注于业务逻辑,使得达到快速开发的目的^[21,22]。

Spring Boot 具备很多优点,例如:

- (1) 基于 Java 快速的创建可以独立的 Spring 应用程序,很容易与主流框架进行集成。
- (2) 提供常见的启动类 Starter 来简化项目的自动依赖加载。
- (3) 使用内嵌的 Tomcat 或 Undertow 等 Servlet 容器,降低了对环境的要求,直接打包为 Jar 包,使用 java 命令即可运行。
- (4) 不需要进行 XML 配置和代码生成,开箱即用。
- (5) 适用于微服务架构^[23]。

Spring Boot 轻量级框架的出现极大的简化项目前期的配置工作,使的开发者从繁重的配置工作中脱身而出,使得系统开发效率得到质的提高。随着微服务时代的到来,各种微服务框架都应运而生,它们不约而同的依赖于 SpringBoot 来构建各个基本应用,因此 SpringBoot 会发挥越来越重要的角色。

2.4 Shiro 安全框架

Shiro 是功能强大且易于使用的一个 Java 安全框架,拥有处理身份验证、授权、加密等功能^[24]。Shiro 提供简单直观的 API,同时也具备强大的安全特征。

在该框架中主要有三个核心概念,主体 (Subject)、安全管理 (SecurityManager) 和领域 (Realm)。主体是一个安全概念术语,其代表任何正在对应用程序执行操作的

实体，既可以代表使用者也可以代表与应用程序之间进行交互的应用程序，比如第三方服务、定时任务等。安全管理类似于一个全局的管理者，协调框架里托管的各个 Shiro 组件及主体对象以确保他们能顺利协同工作。领域扮演将程序的安全加密数据与 Shiro 框架进行交互的角色，当主体执行相关的安全操作时，比如登录或者授权，Shiro 都需要从至少一个 Relams 中得到安全加密数据进而为后续操作提供数据支持。

Shiro 提供了其最主要的身份认证、授权、加密及会话管理四大基石功能，及 Web 支持、缓存、集成测试、多线程应用的并发验证等辅助功能。

身份认证（Authentication），验证用户是否有对应的身份，即通常意义上的登录过程认证。首选获得用户的凭证，该凭证唯一标识该实体，然后将该凭证交给系统处理中，最后通过从 Realms 中获得的数据域凭证进行对比验证根据结果做出相应的决策即可。

授权（Authorization），对用户访问权限进行控制，根据从 Realms 获得的用户角色及权限列表，对访问的资源进行对比验证，根据结果做出相应决策。Shiro 支持通过通配符的方式来进行权限访问描述，因此具有灵活多变的特性^[25]。

加密（Cryptography），数据安全性是整个系统中最重要的一部分，所以安全加密算法的好坏程度直接决定系统的安全系数的高低。Shiro 通过引入对称及非对称加密 API，让用户输入算法名、加密次数及密钥即可进行数据的加解密，极大的简化了用户的使用过程^[26]。

Shiro 的这三大基石功能及额外的辅助特性，使 Shiro 更加简化、灵活易用^[27]。

2.5 Kafka 消息队列

Kafka 是一个开放源代码的分布式事件流平台^[28]。Kafka 自开源以来，因其诸多等关键特性受到许多大型公司的关注和青睐，并将其应用到自己公司的业务中。其主要的适用场景有对数据进行异步处理，通过将对时间不太敏感的多个业务进行分离处理操作，有利于降低系统响应时间，提高系统多并发场景下的性能。

Kafka 使用发布订阅的消息模式^[29]。Kafka 支持多订阅模式，一个主题可以被多个消费者订阅，一个消费者可以订阅多个主题。生产者向对应主题（Topic）发送消息，订阅相应主题的消息者就会收到。其结构如图 2-2 所示。

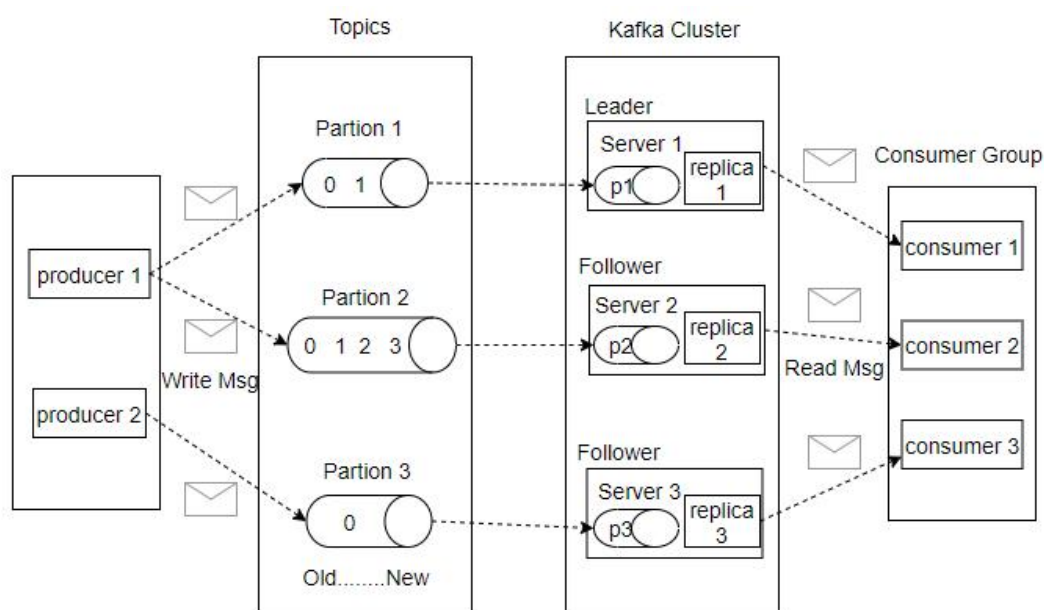


图 2-2 生产消费结构图

简要阐述上图 2-2 的相关概念，Kafka Cluster 表示集群，每个集群有多个 Broker，每个 Broker 表示一个 Kafka 实例。Topic 表示主题，Producer 表示生产者，每次生产者发送消息都需要指定对应主题。同时为了加快消费者消费速度，一个主题可以有多个分区(Partition)，同一主题的消息根据一定的算法分流到不同分区中，常见的有轮训、随机、自定义哈希等算法。一个分区里面的消息是有序的，在某些业务情况下，需要保证消息被有序的消费，此时需要将设置分区数为一来进行保证。主题的一条消息只能被消费者组（Consumer Group）里面的一个消费者（Consumer）消费，但可以被多个消费者组消费。

在本系统的应用中，利用其异步处理机制将与程序主体业务关联不大的操作进行分离，将患者挂号信息及处方信息这一类需要延迟处理的消息进行异步操作，达到到处方的逾期处理及给患者发送就医提醒短信的功能。

2.6 本章小结

本章介绍了企业级框架 Spring、简化数据访问的持久层框架 Mybatis、整合 Spring Boot 框架、进行访问控制的 Shiro 安全层框架及利用异步处理机制的 Kafka 消息队列等技术的基础知识和基本原理，这些技术为后面系统的开发打下了坚实的基础。

3 系统需求分析

系统需求分析是系统开发的首要阶段，需求分析阶段得到的结果是项目开发中设计、编码、测试及维护阶段的重要基础^[30,31]。数据表明，项目开发中遇到的错误发现的越晚，维护成本越高，因此需求分析结果的准确性至关重要。本章从功能及性能等方面对系统的需求做了大致介绍，形成了完善的需求分析文档，为下一步工作指明了方向。

3.1 系统需求概述

医院本着“用心服务从一做起”的服务精神，为了达到患者看病就诊过程更加方便，医生看诊过程更加简单及部门信息及时共享的目的。设计一套集挂号、看病、做检查、缴费、取药等连贯操作的完整的医院门诊管理系统是很有必要性的。该系统作为一种管理工具，实现了一切以病人为起点，从简就医，为就诊病人提供优质服务的目标，这是一种非常人性化的设计。本人通过现场实践走访及与相关医务人员进行商讨，最终确定了该系统涉及的模块的各个流程，随之也确定了每个模块应具备的功能。

门诊病人就诊的第一步为挂号，系统对外展示的功能为在自助挂号平台上展示科室医生的排班信息，列出患者关心的信息，如主要疾病擅长方向，级别信息，等患者比较关注的指标，有助于患者对病挂号就诊。如果是初诊的病人则需要进行注册，由系统匹配相应的卡号，发放给病人，每个病人有一个专有的就诊卡号。当患者挂完号了以后，需要给患者发送短信提醒服务，提醒患者就医。

门诊医生根据患者病情做出诊断，根据病情开出相应的检查、校验、处方项目。所有的项目都需要先到门诊收费平台进行缴费，缴费后的信息会被推送到每个项目对应的科室。如果为检查或校验项目，患者到相应科室进行检查，医技人员对患者做检查后，及时上传检查结果，辅助门诊医生进行确诊判断。如果为处方，患者缴费完成后，系统将主动推送患者的处方信息，等待药房人员配药完成叫号取药即可。

至于系统维护方面，需要有一个系统管理员的角色，该角色作为后台管理人员，对系统拥有最高的权限，对界面上的功能上显示的功能都有操作的权限，包括一般用户具备的功能，比如对账户进行注销和修改操作。同时也需要对一些系统的基本

信息的维护，比如科室信息、医生排班信息、挂号级别信息等进行维护。

3.2 功能需求分析

从上述的需求分析描述中，可以大概的分析出门诊管理系统的主要功能包括用户信息管理、医生排班管理、预约挂号管理、门诊医生工作台、门诊收费管理以及门诊药房管理功能。下面我们从业务的操作流程来描述这六功能模块。

1) 用户信息管理

用户信息管理需要有访问控制和维护用户信息的功能。访问控制包括对用户登录进行身份认证及用户所能访问的系统资源做一定限制；系统管理员作为系统最高的权限拥有者，可以对用户的信息进行修改删除操作，包括对用户的基本资料、角色、科室的修改。

2) 医生排班管理

医生排班管理需要有维护排班计划的功能。当医生需要坐诊时需要提交排班申请信息，然后管理员根据医生疾病专长、坐诊时间、科室进行统一调配。将医生与科室、坐诊时间、挂号级别做对应关系，批量生成医生排班计划供全院患者查看。

3) 预约挂号管理

预约挂号主要的人群是患者，是作为整个 HIS 业务的起始点^[32]。需要有查看科室信息、查看医生排班信息、挂号及取消预约挂号的功能。患者刷就诊卡登录系统后，查询科室信息及该科室下医生的坐诊信息及医生详细信息，然后选择时间进行预约挂号。

4) 门诊医生工作台

门诊医生工作台是对病人看病进行诊断的主要场所，其主要是对患者的病情信息进行诊断、开立检查项、录入就诊信息、开处方的操作。医生在挂号列表选择就诊患者了解基本信息，询问病情详情，然后录入门诊病历，对于不能确定的疾病，需要借助相应的检查或验项目，来协助确诊。对于已确诊的疾病，根据病情开出药品进行治疗。每个处方都有时效性，患者没在指定时间内缴费，则需要做逾期处理。

5) 门诊收费管理

门诊收费工作台是病人就医过程中缴费退费的场所，主要完成收费列表查询、自动核算收费价格、打印收费凭据等功能。当患者到医生站看诊后，医生会根据患

者病情开立相应的检查、检验或药品等收费项目。收费员根据患者就诊卡查看患者待缴费的项目，然后对患者需要进行的项目进行收费，收费同时打印发票。如果患者交费后，未进行相应项目的消费，需要退费则要收回患者相应的发票，同时打印冲红发票。

6) 门诊药房管理

门诊药房工作台是病人就医过程中取药退药的场所，主要完成药品信息维护，门诊处方查询、药品入库、药品发放的功能^[33]。门诊医生在开立完相应的处方，患者到门诊收费处缴费之后，药房工作人员就可以看到相应的处方信息。选择患者已缴费尚未发放的药品，点击发药之后，即可进行发药。

3.3 系统用户及用例分析

从上述需求分析中，我们大概分析出了医院门诊管理系统的角色，主要包括患者、工作人员、系统管理员。患者只要在门诊挂号处注册成功即可为患者。工作人员即在医院从事工作的人员，主要包括门诊医生，门诊药房人员、门诊收费人员等。系统管理员即管理系统的人员，一般由医院的管理层人员组成。角色对应的用例随着用户角色的确定而确定，下面就各个角色用例进行说明。

1) 患者

患者作为医院的主要用户群体，拥有系统界面上显示的操作权限，如用户注册、挂号等功能。其用例图如图 3-1 所示：

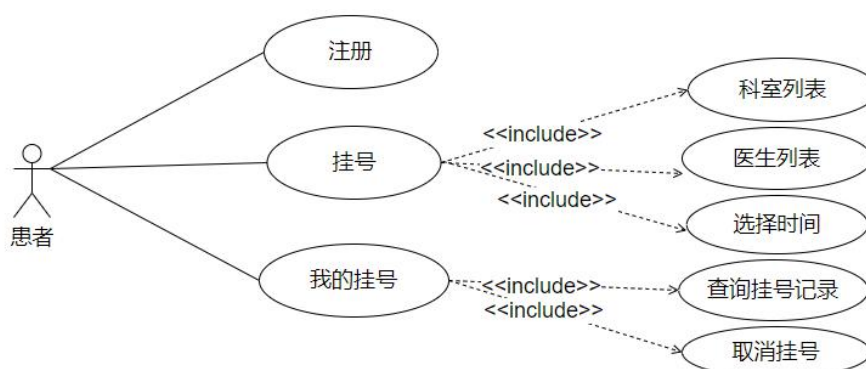


图 3-1 患者用例图

以患者为主要活动的用例说明：

(1) 注册。患者首次进入医院就诊，需要先填写相关必填信息（如患者姓名、身份证号、联系电话等）进行注册。

(2) 挂号。在患者到医院看病的时候首先要进行门诊挂号，挂号前，需要进行科室信息查看、医生信息查看、支付等操作。

(3) 我的挂号。我的挂号的主要内容是保存我的挂号记录。当患者由于挂错了号或者是挂了号没有看诊，此时可以通过取消挂号来进行取消操作。

2) 门诊医生

医生作为给病人看病的对象，其主要是对患者的病情信息的诊断，对相应症状进行开药的操作。其用例图如下图 3-2 所示。

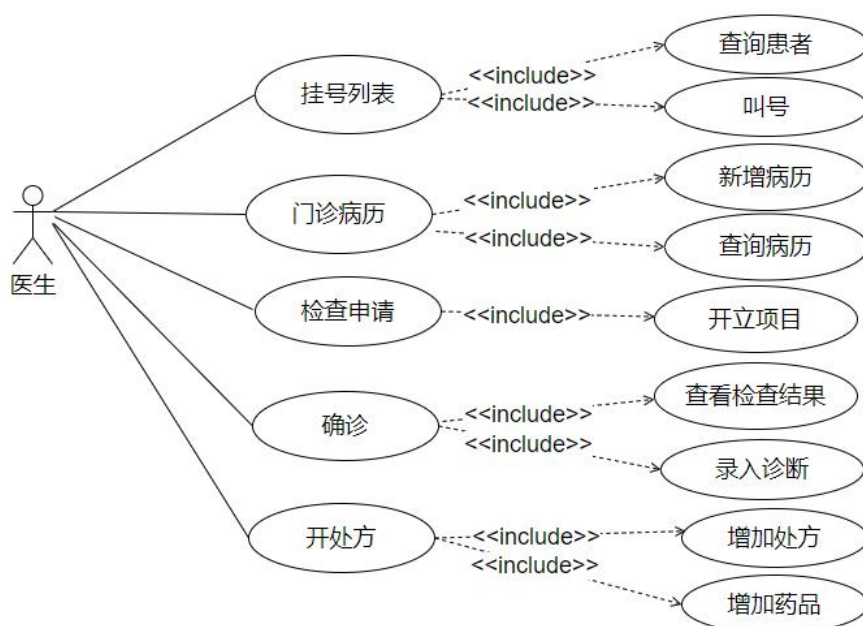


图 3-2 门诊医生用例图

以门诊医生为主要活动的用例图说明：

(1) 挂号列表。在医生诊断主界面，只会显示该医生的所有患者挂号列表，此时可通过点击患者或者通过搜索关键字来进行查找。

(2) 门诊病历。当医生首次接诊患者时，需要询问患者的一些信息，比如患者疾病表现症状，以往的疾病史，当前采取的治疗情况等信息，并将这些信息保存作为病历的一部分。

(3) 检查申请。该功能主要是为医生提供相应的检查申请。当医生填写完病历上关于患者的基本症状信息后，当不能确认患者病情时，需要借助相应的检查、检验项目，来协助确诊。

(4) 确诊。医生先通过平台查看患者的检查结果，当医生对患者的病情能确诊

时，通过该操作完成患者病情的确诊。

(5) 开处方。当医生确定患者的疾病以后，就需要开药治疗。总的来说，处方有成药处方和草药处方两大类。一个处方可以包含多个药品项，医生需要对病开药。

3) 系统管理员

系统管理员作为系统中用户角色权限最高的用户，权限越大，责任也越大。主要是做一些系统信息的维护工作，比如用户注册、维护科室信息、维护排班信息以保障基础信息模块的稳定性。其用例图如下图 3-3 所示。

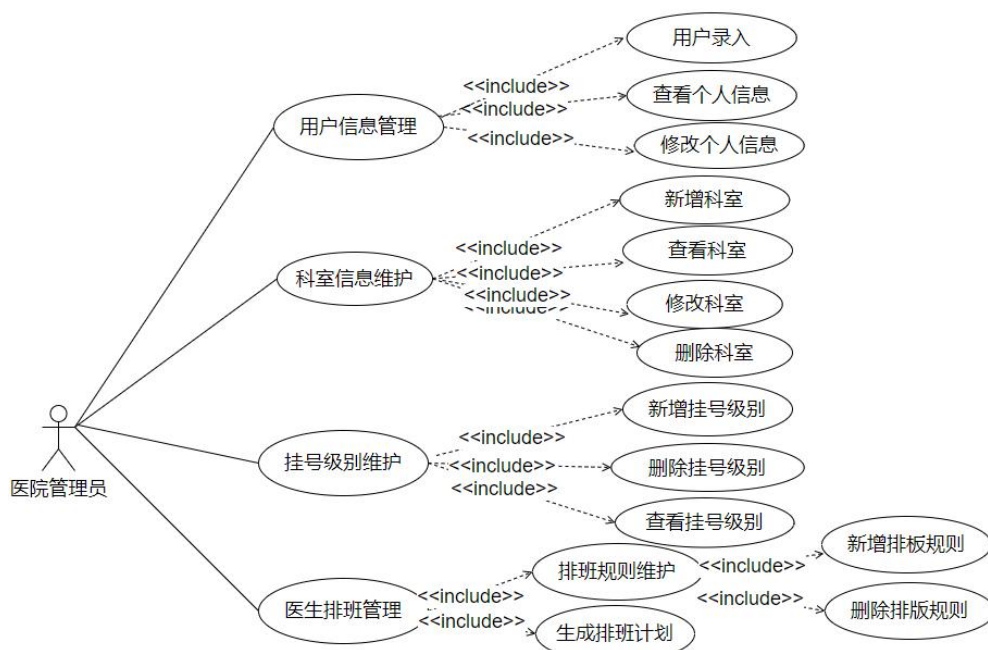


图 3-3 系统管理员用例图

(1) 用户信息管理。用户录入。医院有些信息是需要具有高度准确性保证的，比如医生的信息就不能通过医生自己注册的方式进行录入，其信息能直接影响患者挂号时的判断，所以只有管理员有权限对用户个人数据，例如角色、科室等信息进行统一录入及维护，维护操作包括修改、删除等，其中删除是逻辑上的删除操作。

(2) 科室信息维护。对科室信息的增删改查操作，以确保科室信息的准确性。

(3) 挂号级别维护。用于医院管理员维护医院所使用的挂号级别，以及每种挂号级别的相应挂号费。如普号、急诊号等。分别有增加、修改、查看挂号级别的操作。

(4) 医生排班管理。主要有两类操作，一类是排班计划规则维护，包括排班规则的增加、删除操作。另外一类是生成排班计划操作，生成医生上班时间表。

4) 门诊药房人员

门诊药房人员主要的职责是维护药品的信息，对已经缴费且待取药的患者进行发药操作。其用例图如下图 3-4 所示。

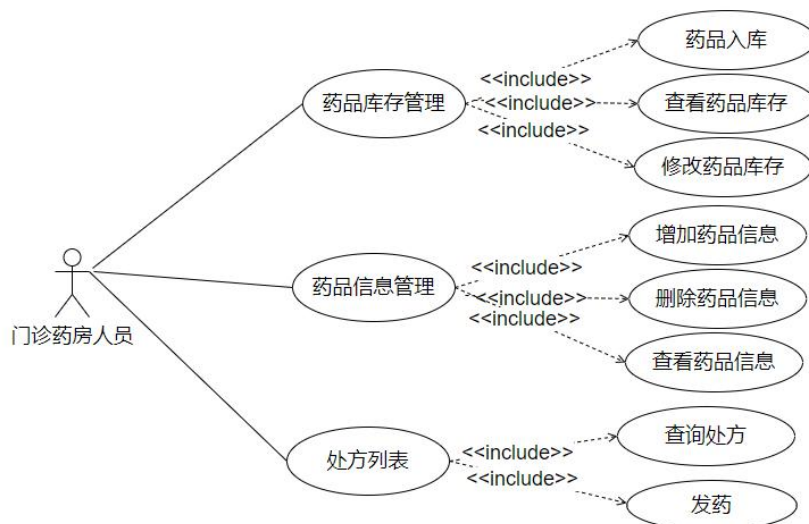


图 3-4 门诊药房人员用例图

以门诊药房人员为主要活动的用例图说明：

(1) 药品库存管理。药房管理员需要维护医院使用的药品的库存信息。主要的操作有药品入库、查看药品库存、修改药品库存操作。

(2) 药品信息管理。药房管理员维护医院使用的药品信息。主要的操作有增加药品信息、删除药品信息及查看药品信息操作。

(3) 处方列表。用于门诊发药人员查找病人的待发药的药品信息，主要有查询患者的待发药的药品信息。对已经缴费的患者按照处方单上面的处方信息进行确认发药操作。

5) 门诊收费人员

门诊工作人员的主要职责是对医生开立的检查、检验或药品等收费项目进行收费，收费的同时打印发票。其用例图如下图 3-5 所示。

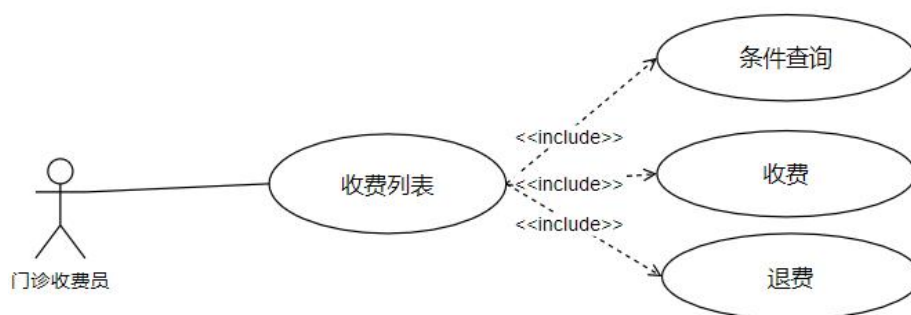


图 3-5 门诊收费员用例图

以门诊药房人员为主要活动的用例图说明：

收费列表。条件查询：根据患者的就诊号进行条件查询，找出该患者未收费状态的项目及预收费总额。收费：确认缴费项目后，点击“收费”按钮，需要录入实收金额，系统自动算出找零金额。点击“确认”按钮，收费结束，打印发票。

6) 医技人员

门诊工作人员的主要职责是对医生开立的检查、检验项目进行检查及将结果上传到系统供医生查看。其用例图如图 3-6 所示。



图 3-6 医技人员用例图

以门诊药房人员为主要活动的用例图说明：

检查。检查：在做检查之前，医技人员先在患者检查界面对需要进行检查的患者进行登记操作，然后对其进行相应的项目检查。检查结果：产生结果后，将检查图片及检查诊断结果通过系统进行上传。

3.4 非功能需求分析

非功能需求在促进软件成功开发的道路上发挥着举足轻重的作用，区别于功能需求界定系统的行为,非功能需求界定系统的质量属性^[34]。作为功能需求的补充，它不描述软件做什么，而是描述软件如何做，往往与系统的总体特性相关。本系统

主要面向患者、医生及管理员，针对系统运行状况和系统自身提出一些要求和标准，包括性能、可靠性、安全性、易用性、可维护性、延展性等^[35]。

1) 性能需求

对于并发访问比较高的模块，如预约挂号模块，系统需要在并发量达到峰值时，提供 800QPS，后端响应数据的时间在 1 秒之内，从请求数据到 Web 页面渲染控制在 2 秒以内是合理的，且对 9 层以上的访问请求都能成功。

2) 可靠性

系统上线后，需要持续不断的提供稳定可靠的服务。一年之中系统的非计划宕机时间需在 12 个小时以内，而且连续 2 次的故障发生之间的时间间隔必须超过 800 个小时，故障切换及恢复正常运行的时间不能超过 15 分钟。发生故障时，用户的数据及业务的逻辑数据不能出现丢失的情况，所以需要按时对数据库数据执行冗余备份操作，降低数据库服务器无故死机导致数据丢失的风险。

3) 数据安全性

此系统涉及的数据都是属于用户的私人数据，如患者的基本信息、患病历史、资料记录等敏感数据，一旦泄漏，将给患者的私人生活带来无穷的困扰。即使是在系统内部，患者的信息需要做一些隔离，比如医生通过系统就诊平台只能看到自己就诊的患者信息。因此对重要的数据如登录密码需要加密存储，对系统不同的用户角色，需要给与不同的访问权限。

4) 易用性

网站界面的展示要统一、人性化风格，达到一看就懂的地步。功能操作要简单，对于业务比较复杂的功能要有比较清晰的导航说明，尽量减少用户跳转页面的次数。

5) 扩展性

在生产环境下需求往往不是一层不变的，系统需要有较强的扩展性，以便以后在业务需求变动及需求增加的情况下做到轻松扩展。设计功能模块时尽量做到模块内高内聚、模块间低耦合，为系统的未来的扩展打好基础，同时需求的变动也不会对已有的代码产生太大的影响。

3.5 本章小结

本章先从现场实践走访体验医院门诊的业务导出项目的整体需求，通过分析得

出系统应具备的功能模块。然后分析就诊过程中涉及的角色，并且对每个角色的具备的操作使用用例图进行阐释。最后对系统的性能、可靠性、数据安全性等这些非功能需求设立了初步的目标。

4 系统设计

4.1 系统总体设计

以上一章节的模块划分为基础，完成系统从整体到局部分层的设计。在整体设计上，从项目架构选型及前后端数据交互方式进行概述，明确所选用的数据交互技术，在局部分层中，划分出各层的交互关系及发挥的职责，让读者对系统涉及有更清晰的认识。

4.1.1 系统体系结构设计

本系统整体上采用当前流行的 B/S 架构，相较于 C/S 架构，虽然 C/S 在架构中数据处理在客户机和服务器端部进行，具有网络通信量低、响应速度快等优点。但是需要同时部署服务器及安装客户端才能运行。在 B/S 架构中数据交互更加轻便，不需要开发客户端，且系统的迭代只需要改变 Web 界面即可实现所有用户的同步更新。所以本系统架构决定采用传统的 B/S 架构^[36,37]。

在选择 B/S 架构的基础上，为了便于开发及维护，本系统采用前后端分离的开发方式^[38]，前后端分别开发及部署，后端服务器提供 Restful API 供前端服务器调用，两者通过 JSON 格式的数据进行交互^[39]。前后端分离的交互图如下图 4-1 所示。

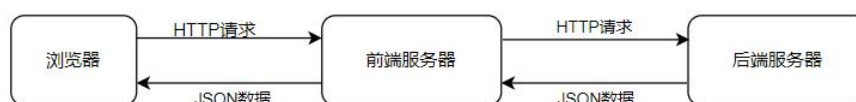


图 4-1 前后端分离交互图

在实际场景中，随着业务需求的增长，为了应对高并发场景也能提供正常的服务，仅仅使用一台服务器是完全不够的，一台服务器的 CPU 处理能力及硬盘存储能力有限。此时更好的一种处理方法为将这些请求压力通过负载均衡软件均匀的发散到不同的服务器上，这样即使面对海量数据请求，通过增加服务器数量即可处理这些请求，同时单个服务器的处理能力也不再成为服务的瓶颈。为了更好的应对高并发问题，本系统后端服务采用集群部署的方式，将后端服务署到多个 Tomcat 容器中，使用 Nginx 来做服务间的负载均衡调用，为了保证 Nginx 的高可用，使用 Keepalived 机制来实现 Nginx 的主备切换^[40]。总体请求架构图如下图 4-2 所示。

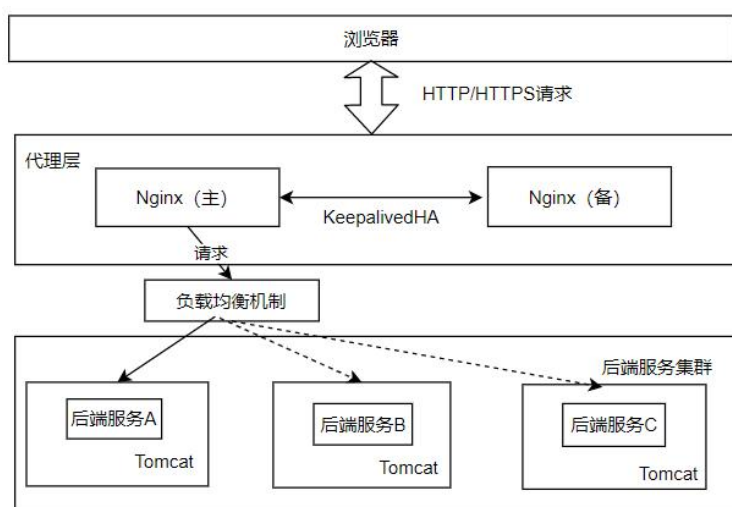


图 4-2 请求调用机制

对总体结构进行规划后，需要根据业界标准对系统展开合理的分层设计，划分出每一层的功能，每一层所包含的模块，然后进行设计即可。从系统分层结构上看，项目可以被分为浏览器层、前端服务器层、Tomcat 服务器层及提供数据支持的数据库服层。系统的分层结构图如下图 4-3 所示。

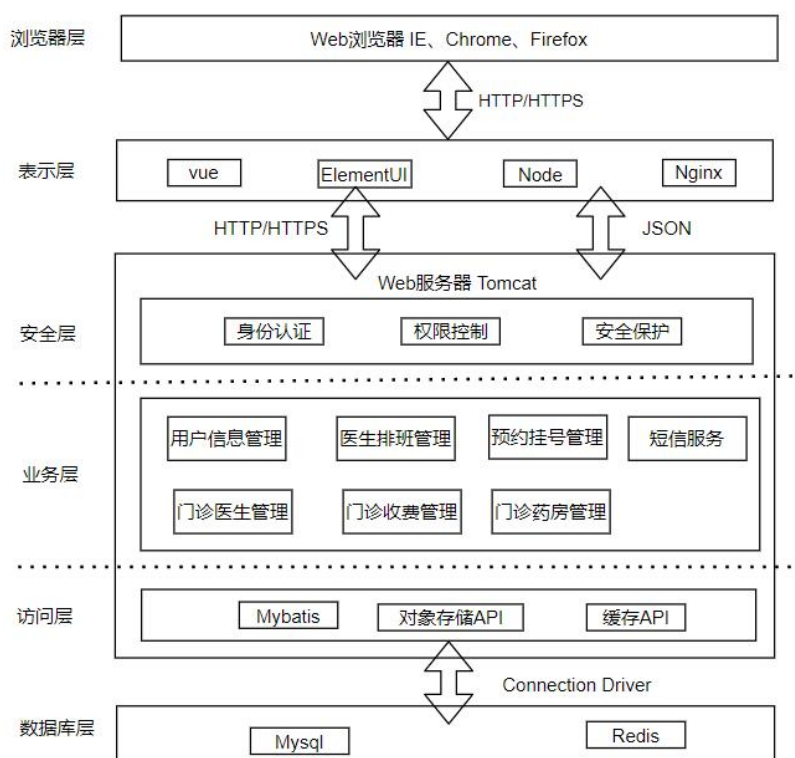


图 4-3 系统分层结构图

根据系统的分层设计图可以看出,系统总共分为四部分,第一部分是浏览器层,浏览器是与用户直接打交道一层,用户可以通过 Chrome/Firefox 浏览器来访问系统,用户的操作都在浏览器页面进行。

第二部分是表示层即前端部分,系统借助 Vue 框架来实现前端页面的开发,使用 Node 启动,同时通过 Webpack 将前端项目管理起来,简化项目的打包操作,打包完成后,将其部署运行在 Nginx 服务器中,利用 Nginx 的高性能、反向代理的特征实现请求负载均衡,有助于充分利用每个服务器的资源,达到资源利用最大化。本系统选择使用 Nginx 运行前端项目,后期后端访问压力紧张而进行扩展时,还可以使用 Nginx 实现负载均衡,为系统升级留下空间。

第三部分是 Web 服务器层,在该层中,首先是安全模块,主要负责是用户登录、用户权限访问起到拦截控制,同时对于一些脚本攻击起到防护作用,然后就是业务逻辑模块,同时也是本系统中最核心的模块,对外提供 Restful 接口的形式提供服务,负责处理业务逻辑,返回结果。访问层也叫 DAO (Data Access Object) 层,简而言之就是与数据库打交道的一层,实现数据的存储操作,对于结构化的数据使用 Mybatis 持久层框架来实现,对于非结构化的数据,使用缓存服务 API 来实现数据的交互。

最后一部分即是数据存储层,一个复杂的系统,一般都有关系及非关系型数据,考虑到 Mysql 的支持事务、支持多并发访问等特点,使用 Mysql 存储涉及的关系型数据,对于热点数据及非关系型数据采用 Redis 来存储^[41]。

4.1.2 系统功能结构设计

从 3.2 节的功能分析和 3.3 用户用例行为分析 2 个章节可以划分得到医院门诊系统的功能结构,其中包括六个功能模块:维护用户信息和用户访问控制的用户信息管理模块;动态调整医生坐诊时间的医生排班管理模块;为患者提供在线预约挂号功能和就医短信提醒的预约挂号管理模块;提供对患者的病情信息进行诊断、开立检查项、录入就诊信息、开处方等操作的门诊医生工作台;完成收费列表查询、自动核算收费价格等功能的门诊收费管理模块;主要完成药品信息维护,门诊处方查询、药品发放等功能的门诊药房管理模块。如下图 4-4 所示,每个模块的具体功能结构将在具体的设计中进行说明。

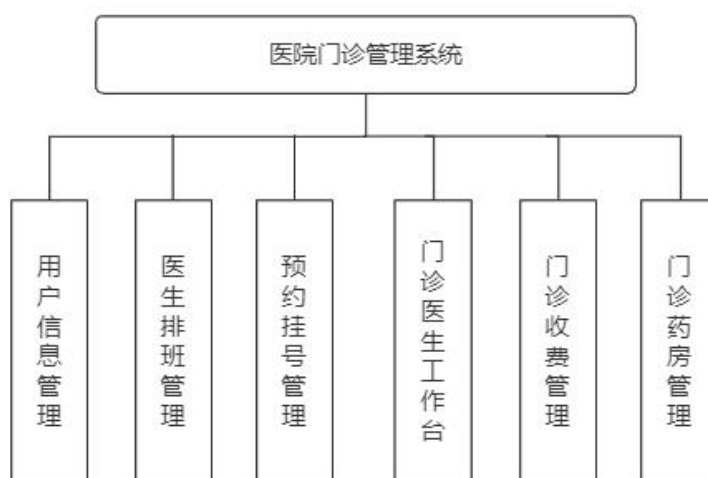


图 4-4 系统总体功能结构图

4.2 系统功能模块设计

门诊管理系统共包含六大模块：用户信息管理、医生排班管理、预约挂号管理、门诊医生工作台、门诊收费管理、门诊药房管理。下面将对这 6 大功能模块进行详细说明。

4.2.1 用户信息管理

用户信息管理模块主要是对资源访问进行控制和维护用户的信息，访问控制包括用户登录、添加角色操作，维护用户信息主要包括常见的查看、修改、删除、添加操作，功能结构如图 4-5 所示。

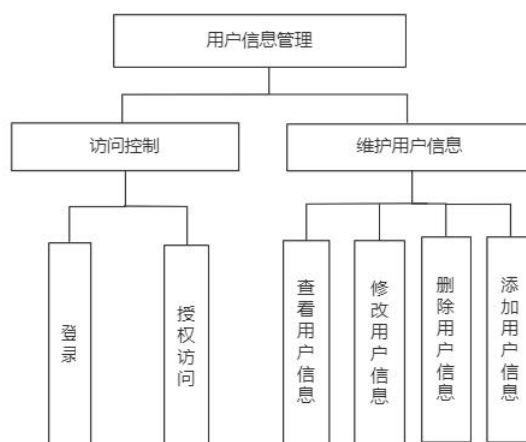


图 4-5 用户信息管理结构图

登录。用户访问系统的第一步就是输入用自己的信息，比如注册用户名和密码，来进行身份认证。通过后，才能对具有一些模块的操作权限，这样才能起到一定的保护作用，提高系统的安全性。用户登录后，才能在个人中心模块对自己的信息进行维护。在用户身份认证过后，将用户的身份 Token 存入 Redis 缓存中。用户登录的流程图如图 4-6 所示。

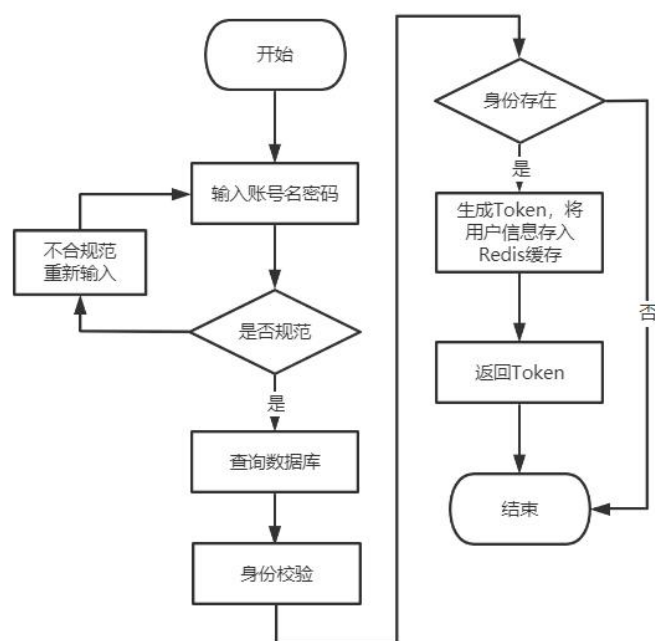


图 4-6 用户登录流程图

授权访问。授权访问主要是对用户访问系统的资源时对用户的身份、所具有的操作权限与数据库的已存的安全数据的对比（包括加密后的用户密码、用户具有的角色及权限列表）的对比校验，其大致流程如下图 4-7 所示。

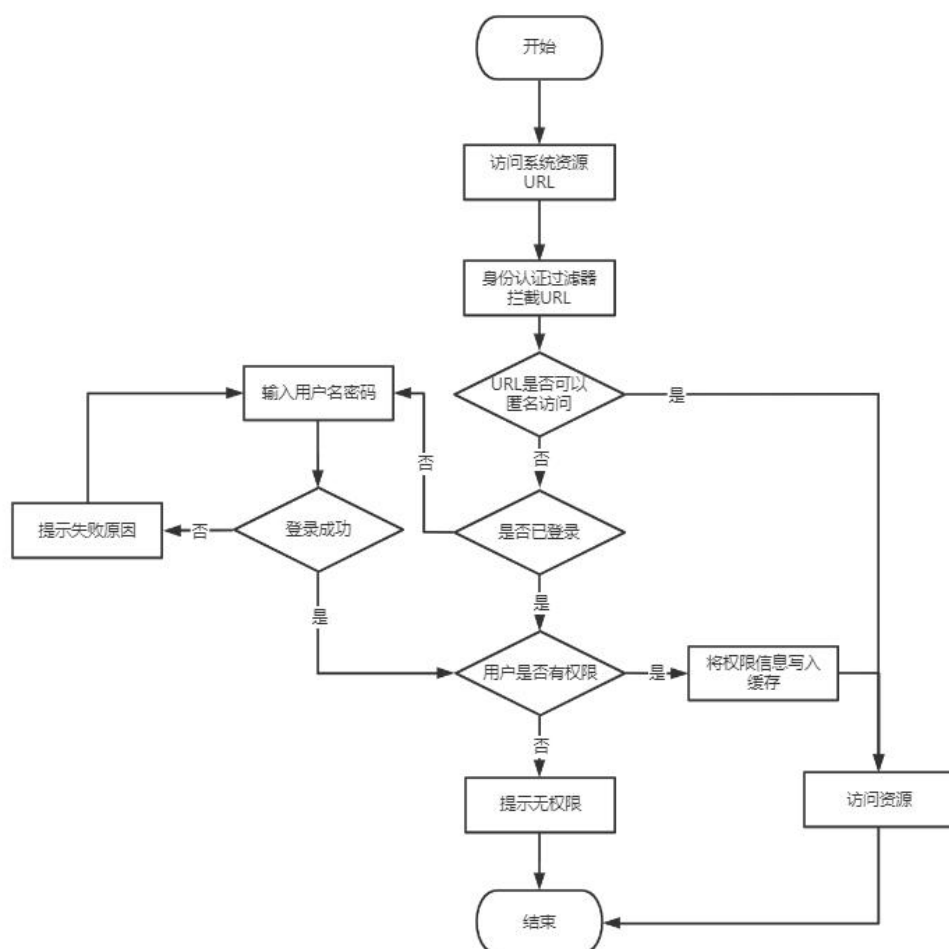


图 4-7 授权访问流程图

维护用户信息。用户登录系统了以后，在个人中心可以看到自己的具体信息，比如姓名、头像等。如果用户，比如门诊医生、收费员离职，为了避免不必要的风险，管理员需要将对应的账号给注销，注销的账号不能再登入系统。

本系统采用 Shiro 来实现整体安全层的控制，使用基于角色的访问控制 RBAC (Role Based Access Control) 模型来实现用户与角色，角色与权限的匹配^[42]。用户系统支持系统管理员管理用户的基本资料，包括对用户的基本资料及用户的角色信息的修改，用户的基本资料包含有用户名、密码、科室等信息，用户匹配的角色有管理员、门诊医生、门诊收费员等。管理员修改用户信息的流程如下图 4-8 所示：

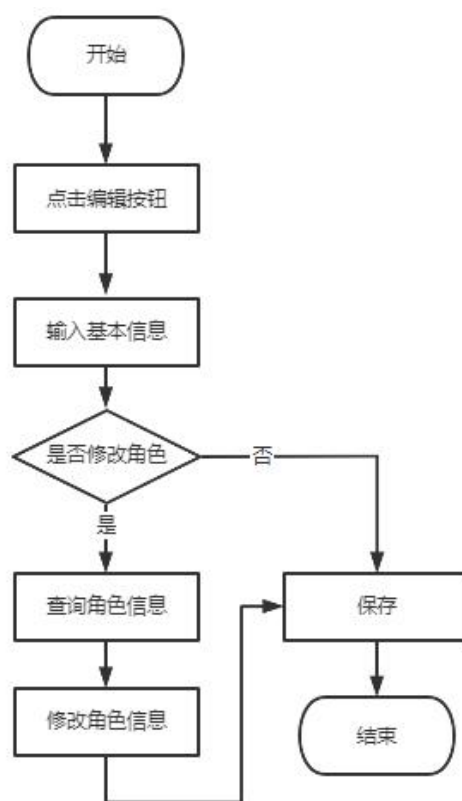


图 4-8 用户信息修改流程图

4.3.2 医生排班管理

医生排班管理模块的功能是维护医生坐诊上班的时间，对医生的上班时间进行规划，只有管理员有权限进行操作。其主要包含查询、新增、删除、修改排班计划，其功能结构如图 4-9 所示。

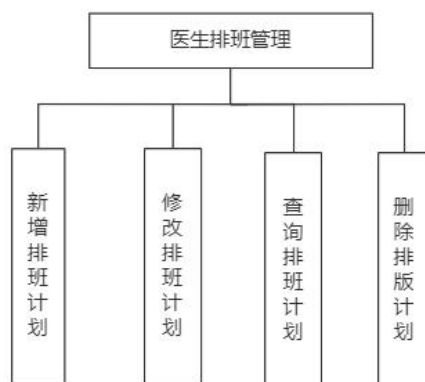


图 4-9 医生排班管理结构图

查询排班计划。系统管理员进入医生排班页面，通过下拉框进行科室选择，此时的下拉框会隐含式的调用信息管理模块的科室查询接口，返回科室的名称填充页面。然后选择科室，选择查询标签，根据科室条件调用排班模块的条件查询接口，得到医生排班信息进行回显填充页面展示医生的排班信息。

新增排班计划。此功能主要是针对排班管理员，主要的功能是增加医生排班规则信息。系统管理员登录系统，选择新增排班标签，跳转到新增排班界面。通过科室选择下拉框选择科室信息，根据科室条件调用排班模块的条件查询接口，得到医生排班信息。最后批量选择医生和排班时间，填写排班规则名称，排班规则名称由前端进行校验，通过后，提交医生排班信息到排班管理模块的医生排班规则新增接口。调用系统接口时，由权限控制模块进行用户权限校验，如果没有权限提示用户权限不足，如果有权限直接调用排班管理模块的新增排班接口，根据数据库新增返回信息，返回相应的成功或失败信息。修改排班计划与新增排班计划流程类似，直接对已有数据进行覆盖即可。增加排班规则的流程图如下图 4-10 所示。

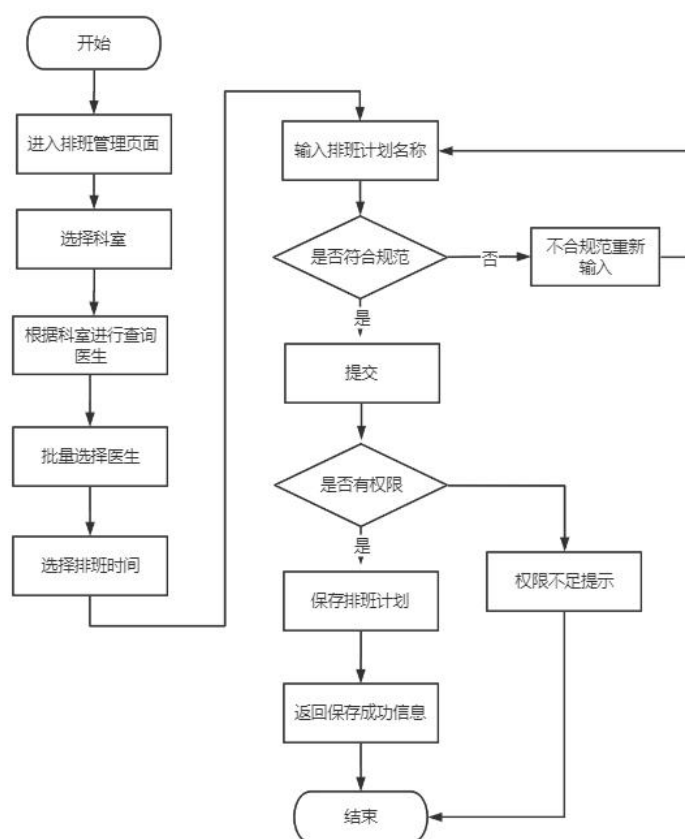


图 4-10 新增排班规则流程图

4.3.3 预约挂号管理

预约挂号管理模块的功能是提供在线预约挂号服务。其主要功能分别为：查看科室医生坐诊详情、挂号及对就医患者进行短信提醒服务。其功能结构图如图 4-11 所示。

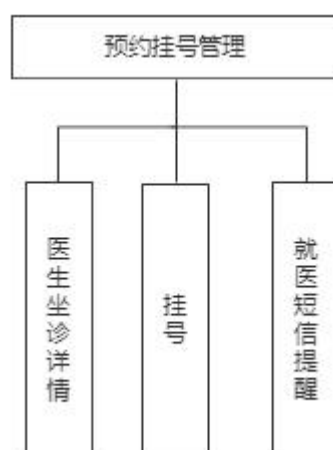


图 4-11 预约挂号结构图

查看科室医生坐诊详情。在进行挂号之前，患者需要先了解科室下医生详细信息，包含主要疾病擅长方向，级别信息，挂号费用等患者比较关注的指标，有助于患者对病挂号就诊。当患者查看某个科室的医生信息时，调用医生排班模块的查看医生排班信息 API，得到医生的坐诊信息。由于医生排版模块的返回的信息只有医生的排班信息，此时需要调用用户信息模块的查看用户具体信息接口，得到医生详细的信息。然后将返回数据填充医生挂号页面即可。查看科室医生坐诊详情时序图如下图 4-12 所示。

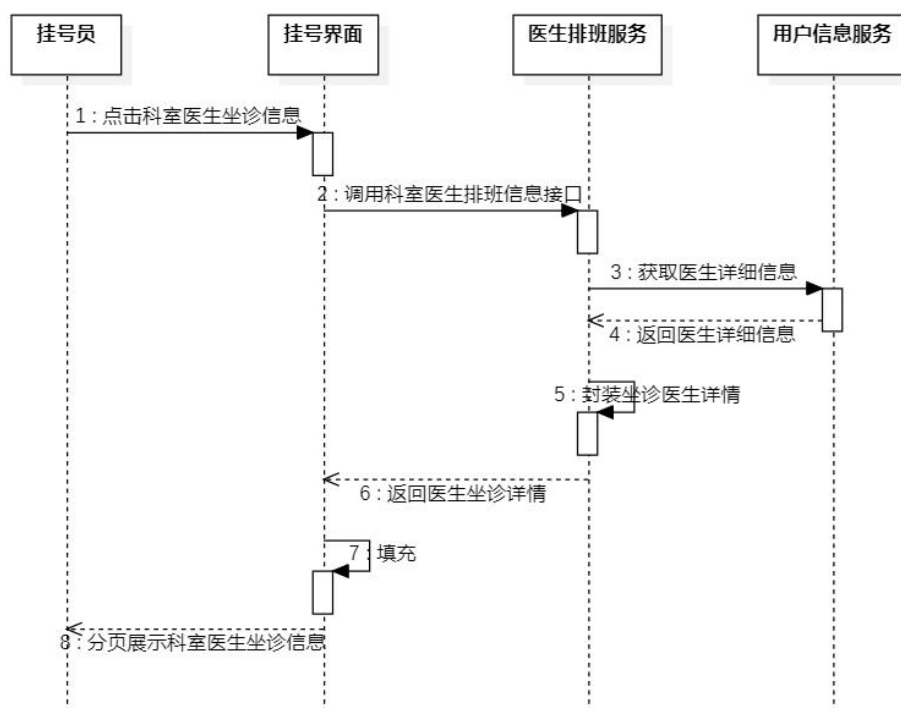


图 4-12 查看科室医生坐诊详情时序图

挂号。挂号是来院就诊的第一个环节，完成挂号后才能继续后序的就诊、缴费、检查、拿药等环节。当患者选中医生后，挂号页面点击挂号标签，携带用户信息及医生信息向挂号服务发送挂号请求。此时由于传递的用户信息只有患者就诊卡号，需要携带患者信息向用户模块发送查询请求，得到患者的详细信息。然后构建患者挂号对象，且将此挂号医生的挂号人数减一。构建完成后，需要保存此挂号就诊信息，调用此模块的保存就诊挂号服务保存就诊挂号信息。最后需要将此次的收费状态记录保存下来，此时就需要调用账单服务模块的服务，将收费状态记录保存到数据库即完成了整个挂号流程。挂号业务时序图如下图 4-13 所示。

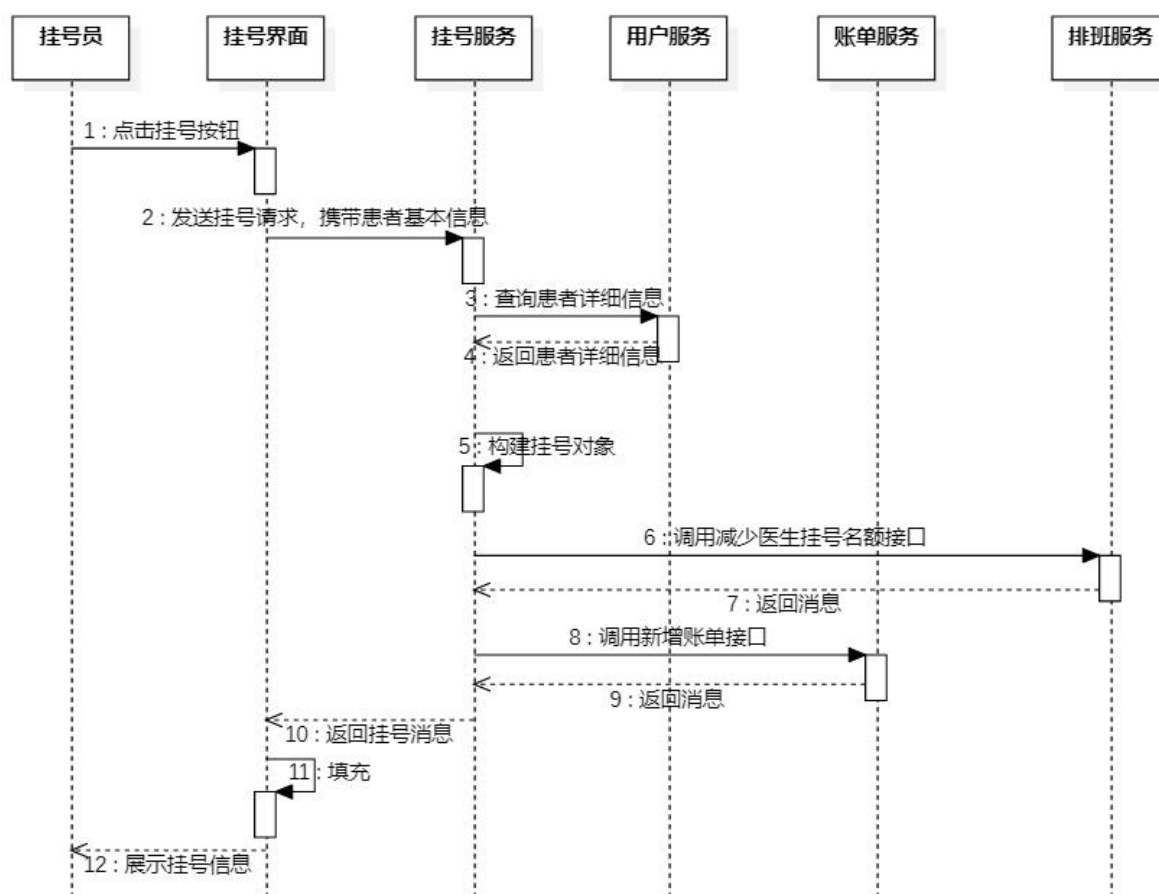


图 4-13 挂号业务时序图

就医短信提醒。现今社会的很多人面临来自生活各方面的压力，进而很多事情都容易遗漏，看病也是如此。患者可能今天挂的后天的某科室的专家号，到了后天却由于某些事情忘记了挂号这一事情，错过了看诊时间，导致错过了最佳看诊时期，所以此时给患者来一个当日就诊短信提醒是非常有必要的。本模块使用定时任务在每天早上 7 点的时候，遍历患者的挂号列表，对于看诊日期是今日的挂号列表，取出挂号记录，将其放入消息队列 *Kafka* 的就医提醒主题中，这个主题中里面的消息会被延迟消费。到了预定的延迟时间后，监听这个队列的消费者则消费且解析其中的消息，得到患者的挂号信息，比如手机号，挂号科室等，然后调用阿里短信服务给患者发送就医提醒短信。就医提醒短信的时序图如下图 4-14 所示。

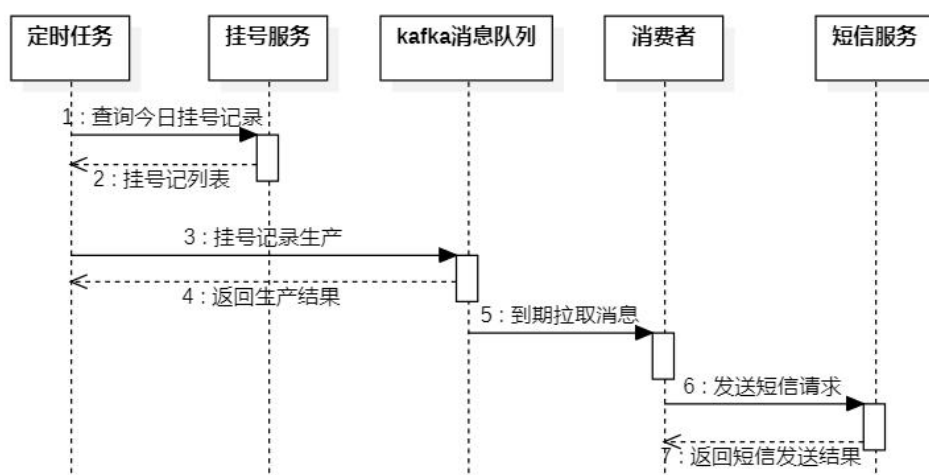


图 4-14 就医提醒短信的时序图

4.3.4 门诊医生工作台

门诊医生工作台是医生工作地，也是病人看病的主要场所。其主要有门诊诊断、开立检查项、录入就诊信息、开处方等操作。其功能结构图如下图 4-15 所示。

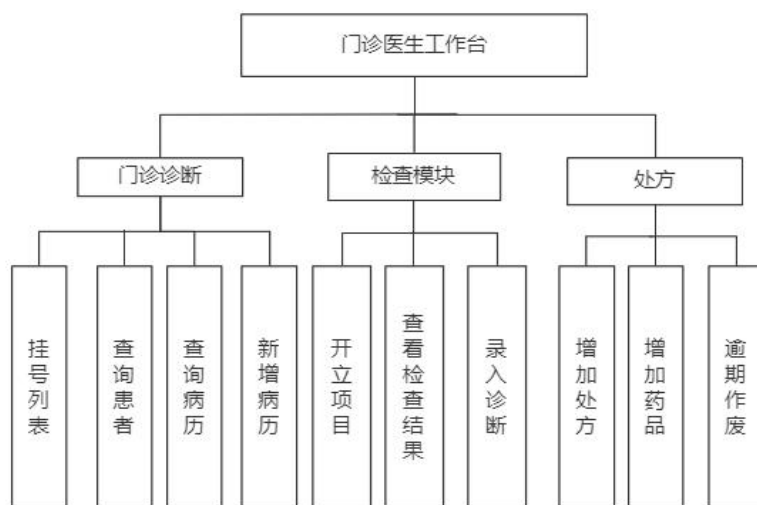


图 4-15 门诊医生工作台功能结构图

门诊诊断。门诊诊断主要包括医生登录系统查看自己诊断的患者的信息及为患者添加诊断信息的流程。当医生登录系统后，需要在门诊医生工作台首页看到自己就诊或已挂号的患者信息，有了这些信息以后才方便后序的查看患者信息及根据这些信息对患者进行搜索。此时就需要携带自己的身份令牌 Token 向诊断病人管理模块发送请求查询诊断患者信息。患者来到医生就诊处后，医生需要拿到该患者的就诊详细信息才可以进行下一步的诊断操作，此时医生点击左边栏用户列表，然后根

据这个用户信息调用病历管理模块的查询患者就诊信息 API 拿到患者就诊病历信息。对于初诊的患者，其诊断病历默认为空。对于复诊的患者，默认取其最新的一条诊断记录。最后医生对病人开始诊断，询问患者病情，在病历页面进行填写患者病情信息保存即可。门诊诊断的流程图如下图 4-16 所示。

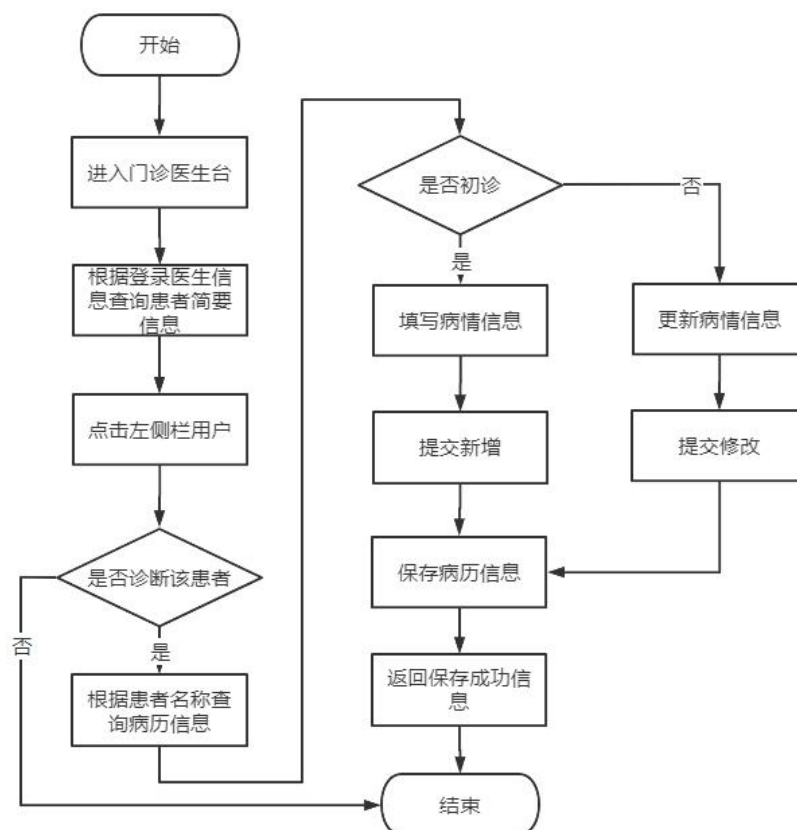


图 4-16 门诊诊断流程图

检查模块。当医生填写完病历上关于患者的基本症状信息后，当不能确认患者病情时，需要借助相应的检查、检验项目，来协助确诊。门诊医生进入检查申请界面，选择新增项目，在项目选择列表选择相应的检查项目，点击检查项目添加即可，然后需要在开立检查之前，在项目后面填写检查要求，最后点击开立即可。在医技人员检查完患者，填写相应的诊断结果及上传相应的照片后。门诊医生即可在下次患者复诊的时候看到已开立的检查项目的结果。该模块时序图如下图 4-17 所示。

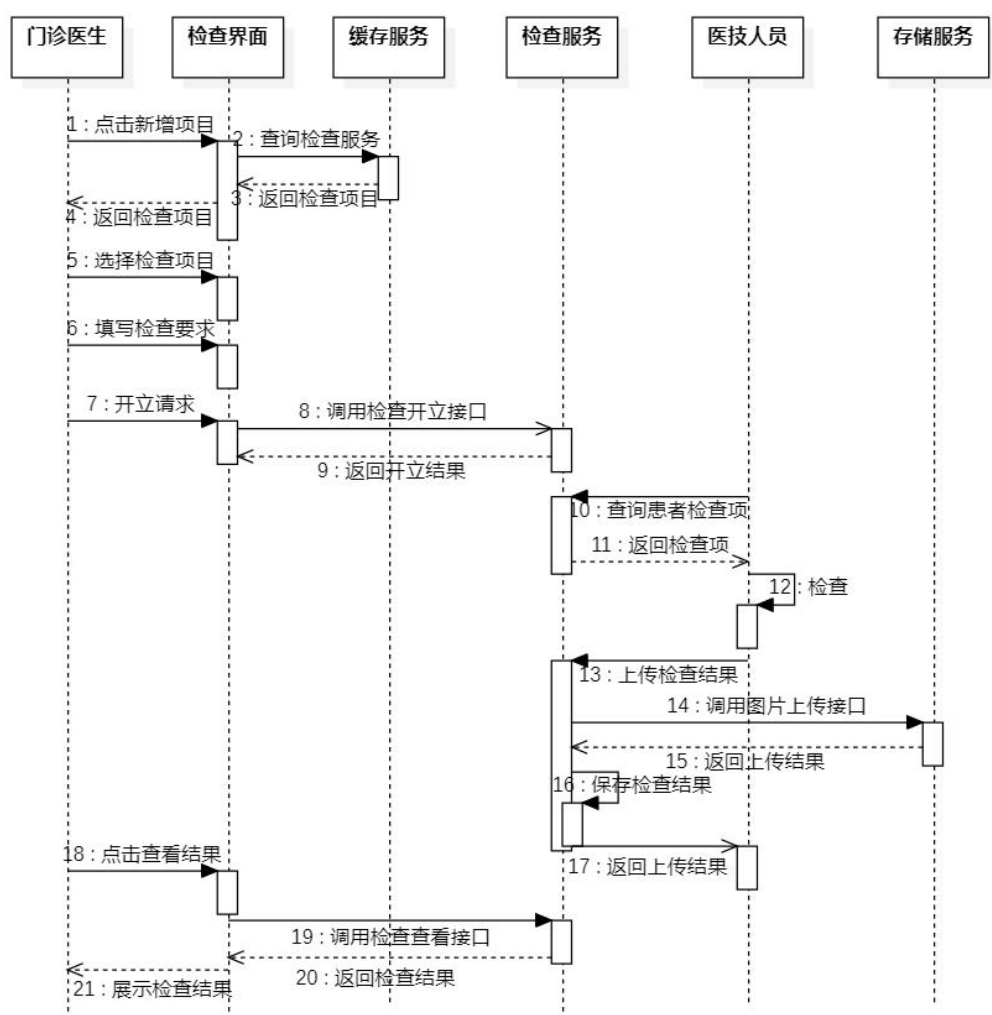


图 4-17 检查模块的时序图

处方。当医生确定患者的疾病以后，就需要开药治疗。在增方页面选择增方标签，输入处方名称。根据患者病情需要选择添加相应的药品，点击添药标签。由于药品的名称比较不易记忆，在药品添加页面，输入药品查询条件，借助药品搜索服务拿到药品信息。选中药品后，输入此药品的使用事项，点击添加完成该药品添加操作。在处方提交之前，根据国家规定，每个处方中最多只能包含 5 种药品，如果超过则给出处方包含药品数量超出阈值警告。最后提交该处方信息，向处方管理模块发起开立处方保存请求，保存处方信息即可。开处方的流程图如下图 4-18 所示。

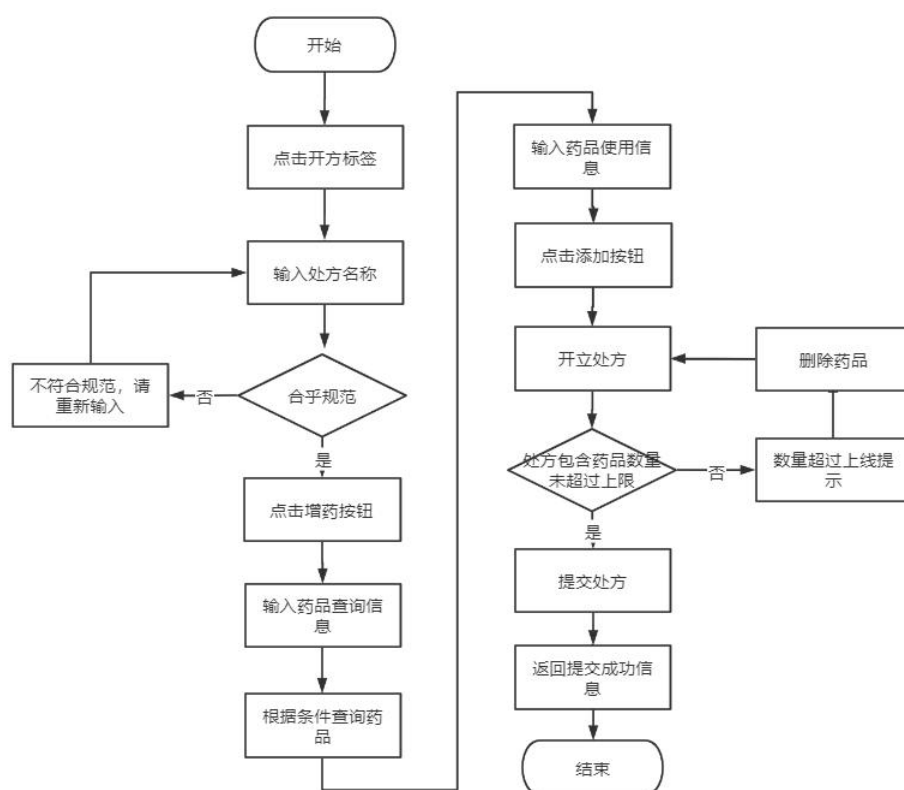


图 4-18 开处方的流程图

逾期处理。当完成处方开方以后，处方具有时效性，患者一段时间内不缴费的话，需要将处方从可用状态改变为已过期状态，设置为过期状态的处方，门诊收费部门是查询不到此处方信息的。在处方开放完成后，将处方的相关信息传递到 Kafka 某一主题中，这个主题里面的消息会被延迟消费。到了设置的时效性之后，该主题的消息就会被监听这一主题的消费者所消费。消费者消费其中的消息，调用处方管理模块的 API 拿到此处方信息，根据其状态做相应的处理。处方超时未缴费过期处理的时序图如图 4-19 所示。

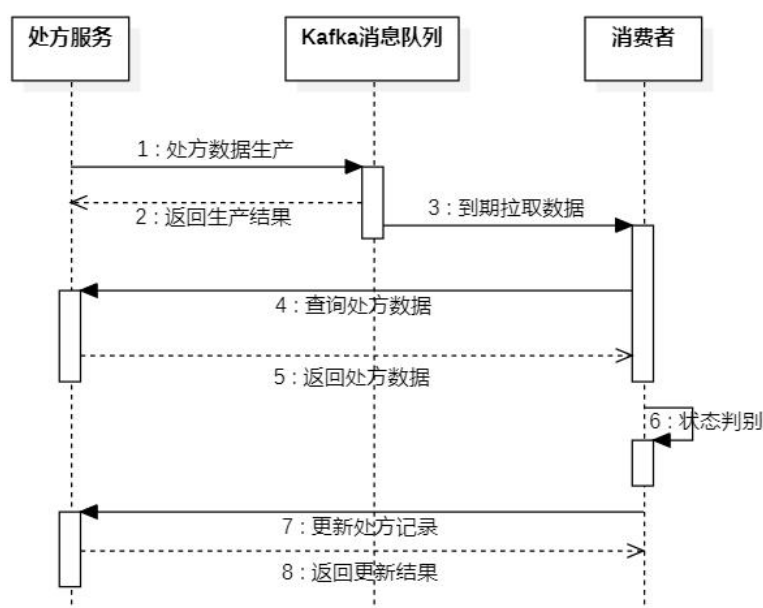


图 4-19 处方超时未交费过期处理时序图

4.3.5 门诊收费管理

门诊收费管理主要完成收费列表查询、自动核算收费价格、收费、打印收费凭据等功能。收费管理是其中非常重要的模块，所以需要在设计过程中充分考虑每一个具体细节的实现，确保整个模块能稳定的运行。其功能结构图如下图 4-20 所示。

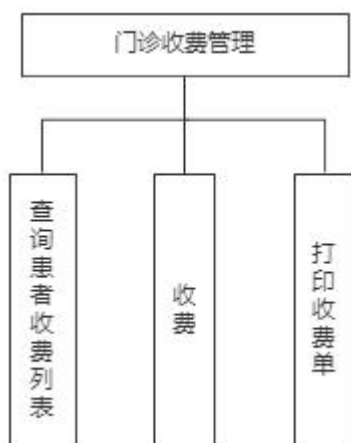


图 4-20 门诊收费管理结构图

查询患者收费列表。患者到挂号处挂号及门诊医生站看诊后，会产生相应挂号费用及医生会根据患者病情开立相应的处方药品收费项目。当患者来到收费处后，

门诊收费员需要先得到患者的信息，此时刷患者的就诊卡，通过患者的就诊卡号调用患者信息模块的患者查询接口得到患者的信息。一个患者可能有多个挂号记录，所以需要通过患者的 ID 调用挂号管理模块的查询接口得到患者的挂号列表。因为无论是挂号还是医生开处方，设计的时候都会调用此模块的账单新增接口，存储相关的账单记录以便日后使用。此时可以遍历患者的挂号列表，通过患者的挂号 ID 调用此模块的账单查询接口，得到账单的详情。向用户展示的时候需要展示账单包含的具体项目，需要通过账单包含项目 ID 调用相关模块的项目查询接口。如处方模块，就需要调用处方模块的根据处方 ID 查询处方详情的接口，获取处方中的处方项详情，比如处方包含处方项名称、处方项金额等信息。查询患者收费列表的时序图如下图 4-21 所示。

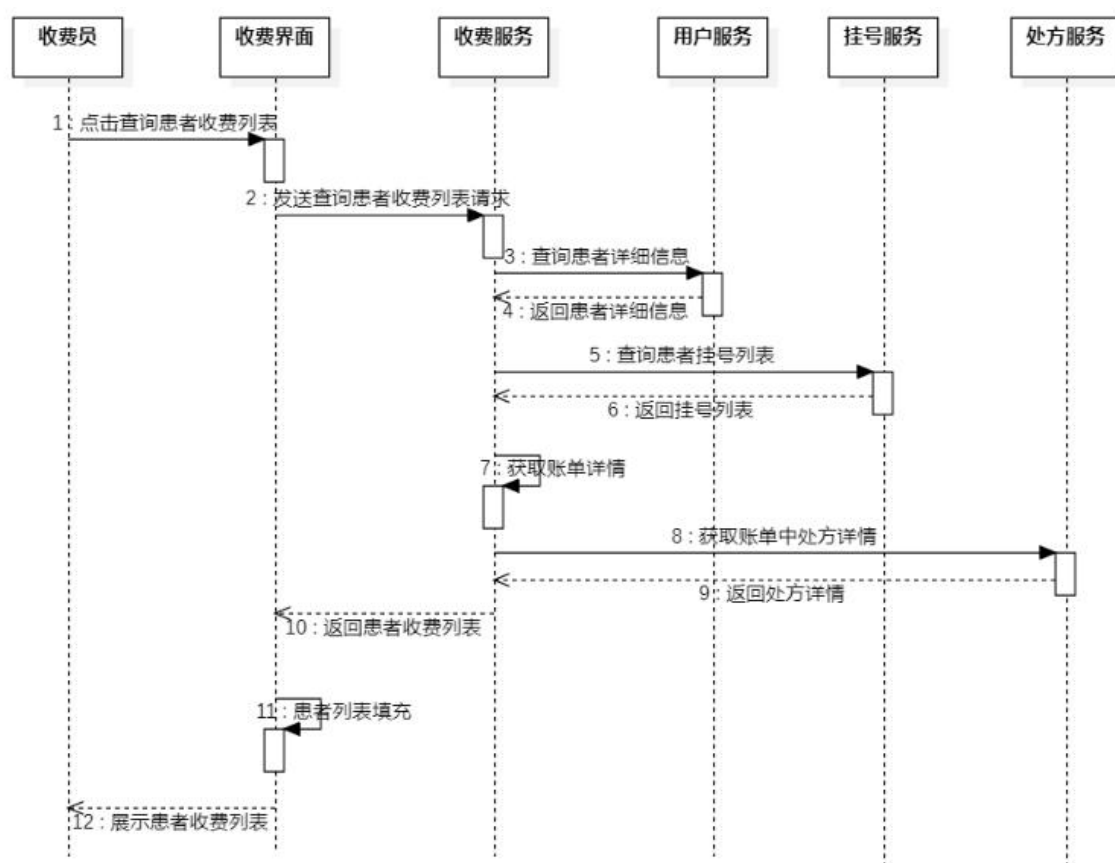


图 4-21 查询患者收费列表时序图

收费。当患者确认要缴费的项目后，系统根据所选的收费项目自动核算出需要缴费的金额，选择缴费标签时，会让你输入收费的金额及系统自动根据收费的金额

减去应收金额得出需要找的零头。如果零头小于 0，那么提示请输入正确的缴费金额。选择确认标签，调用服务器该模块的项目收费方法，存储收费的项目。最后一还需要将此次的交易记录记入数据库，调用该模块的发票新增方法，根据数据库新增返回信息，返回相应的收费成功或失败信息。门诊收费管理的流程图如下图 4-22 所示。

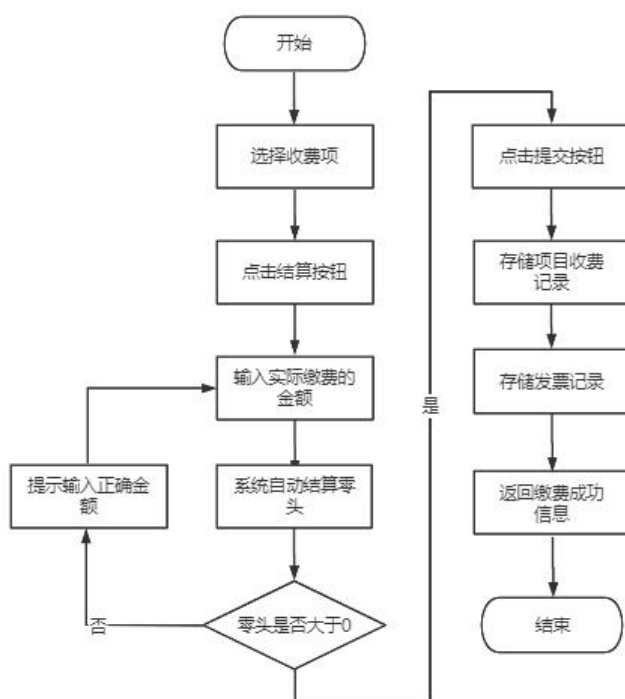


图 4-22 门诊收费流程图

4.3.6 门诊药房管理

门诊药房管理是门诊药房实现信息化管理的一种工具，主要完成药品信息维护、查询发药患者列表、发药的功能。其功能结构图如下图 4-23 所示。

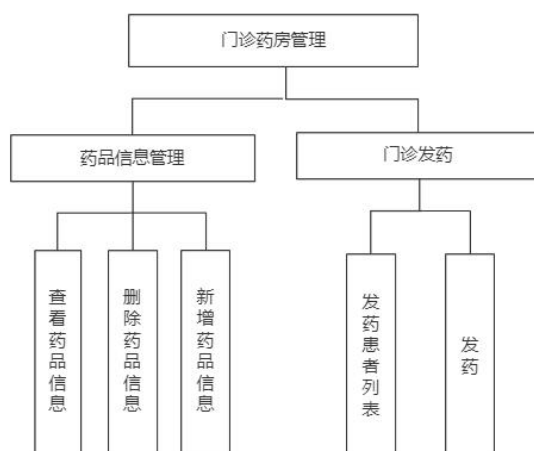


图 4-23 门诊药房功能结构图

药品信息管理。医院新引进了一批药品，需要在药房存储里面批量的添加。如果某种药品由于种种原因，医院不再使用，那么就需要对其进行下架，以免医生查询到此药再次开药，避免后期不必要的退款操作。在界面上展示药品信息时会调用此模块的查看所有药品方法获取药品信息，调用方法的同时会传递自身登录时得到的身份令牌 Token。在真正拿到药品信息前，鉴权服务使用该 Token 及调用用户服务查询用户角色及权限信息进行权限校验。通过校验后，则获取所有药品信息，并且以分页的形式返回，在药房管理界面展示。药品列表的时序图如下图 4-24 所示。

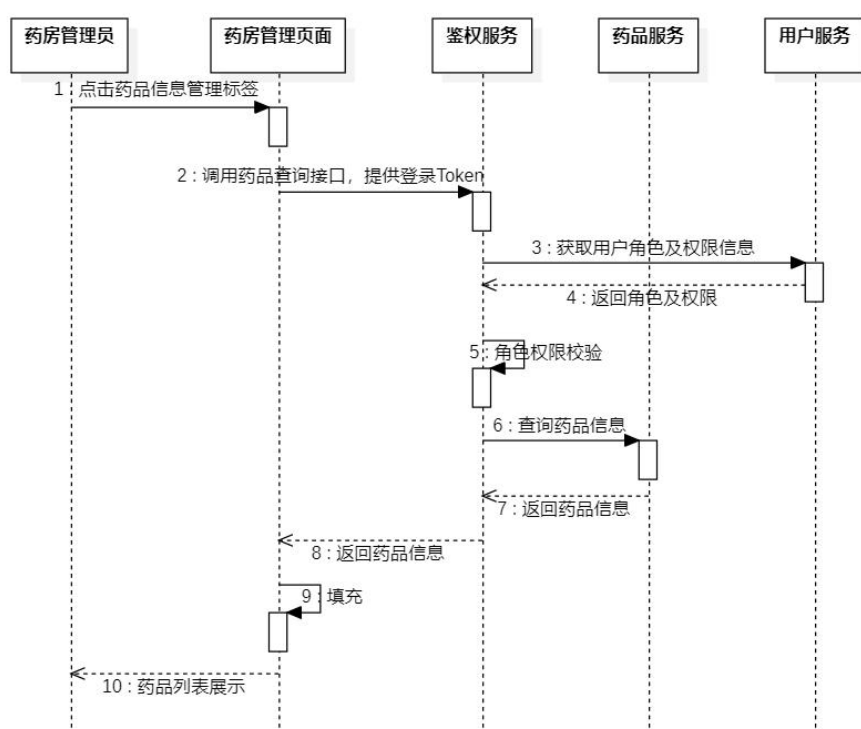


图 4-24 药品列表时序图

药品管理页面的新增功能主要是针对系统中没有的数据，在药品添加页面，药品的主要信息有编码、名称、规格、单价等。填写完毕后，选择保存标签，再真正保存之前，由于设置的药品编码是为唯一属性，如果数据库已存在此药品编号，则不能添加此药品，保存失败。如果不存在，则将新添加的药品信息保存到数据库中。

门诊发药。此模块有两部分，一部分展示发药患者列表及发药患者具体的发药信息，药房管理员进入药房工作台页面后，需要看见发药患者列表信息。此时调用药房工作台模块的查看发药患者列表接口，拿到未发药及已发药的患者信息。当患者取药时，药房工作人员需要输入查询条件如患者病历号或者姓名，调用药房工作台模块的查看患者发药详情接口，拿到患者的发药信息填充到页面的发药栏。另外一部分是发药，对选中的处方药品列表，进行发药，将选中的处方药品信息调用药房工作台模块的患者发药接口，由其实现类实现药品发放信息的存储。

4.3 系统数据库设计

关系型数据库经过多年的发展，在开发存储中一直占据重要的位置。本系统采用 MySQL 存储系统涉及数据，如患者就诊数据，门诊医生看诊数据，处方数据，

诊断病历等。

4.3.1 系统 E-R 模型

E-R 模型表示实体关系模型，反映了系统中实体之间的关系。本系统包含的实体主要有：患者、门诊医生、挂号记录、病历、处方、账单记录、排班计划等实体。具体的 E-R 图如下图 4-25 所示。

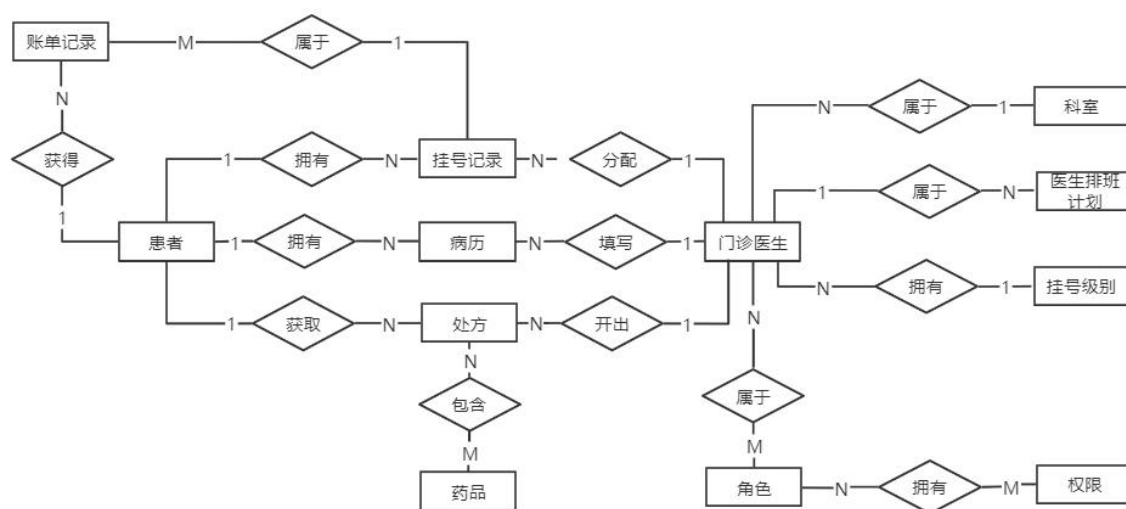


图 4-25 数据库 E-R 图

门诊医生与处方之间是一对多的关系，一名医生可以开很多处方，一个处方只能由某个医生开立。患者与处方也是一对多的关系，一名患者可能多次就诊，因此会存在多个处方记录，而一个处方只能针对特定患者的特定疾病进行描述。在系统权限管理部分，使用 RBAC 的设计思想，设计了用户表、角色表、权限表，用户和角色之间、角色和权限之间使用连接表进行关联，均属于多对多的关系，单个用户可以分配有很多角色，每个角色有独立的权限组，这样就可以灵活地进行系统权限控制。

4.3.2 MySQL 表设计

该系统的表结构主要以患者为中心，主要的表的字段都是与医生对患者的诊断操作有莫大的联系，本系统涉及的表的数目众多，多达 25 张表，这里仅列出其中比较重要的表的结构。

1) 患者信息表。该表记录了本系统患者的个人信息，患者不需要登入系统，因此没有用户名和密码等字段。包括患者姓名、地址、电话号码、就诊卡号等字段。

表结构如表 4-1。

表 4-1 患者信息表

字段名	数据类型	约束	备注
patient_id	bigint	主键	患者 ID
patient_name	varchar	非空	患者姓名
gender	varchar	非空	用户性别
birthday	datetime	非空	出生日期
ID_card	varchar	非空	身份证号
medical_card	varchar	非空	就诊卡号
phone	varchar		联系电话

2) 用户信息表。该表用于记录使用本系统的用户的个人信息，包括用户 ID、角色 ID、部门 ID、挂号级别 ID、用户职位、疾病专长等字段。表结构如表 4-2。

表 4-2 用户信息表

字段名	数据类型	约束	备注
user_id	bigint	主键	用户 id
username	varchar	非空	用户名
password	varchar	非空	用户密码
gender	varchar	非空	用户性别
phone	varchar	非空	联系电话
dept_id	int	外键	用户科室
registration_rank_id	bigint	外键	挂号级别
position	varchar	非空	用户职位
expertise	varchar		疾病专长

3) 挂号记录表。该表用于记录患者的挂号信息，包括挂号 ID、挂号患者 ID、挂号医生 ID、就诊状态、付款状态等信息。表结构如下表 4-3 所示。

表 4-3 挂号信息表

字段名	数据类型	约束	备注
registration_id	bigint	主键	挂号 ID
patient_id	bigint	外键	患者 ID
patient_name	varchar	非空	患者姓名
user_id	bigint	外键	挂号医生 ID
create_time	datetime	非空	挂号时间
status	int	非空	就诊状态
pay_status	int	非空	付款状态
operator_id	bigint	外键	挂号员 ID
jiuzheng_time	datetime	非空	就诊时间

4) 病历表。该表记录了门诊患者的病历信息，包括患者疾病主诉、现病史、发病时间、初诊结果串、体格检查串（做了哪些检查）等。表结构如下表 4-4 所示。

表 4-4 患者病历表

字段名	数据类型	约束	备注
case_history_id	bigint	主键	病历 ID
patient_id	bigint	外键	患者 ID
create_user_id	bigint	外键	诊断医生 ID
chief_complaint	varchar		主诉
his_of_pre_illness	varchar		现病史
his_of_treatment	varchar		治疗史
start_date	datetime	非空	发病时间
pril_dise_str_list	varchar		初诊串
check_str_list	varchar		体格检查串
check_result	varchar		检查结果
def_dise_des	varchar		确诊描述

5) 处方表。该表记录了医生所开处方的信息，包括处方 ID、挂号 ID、开方医生 ID、处方总费用、处方名、使用事项、处方领取状态、处方付款、完成状态状态。表格结构如下表 4-5 所示。

表 4-5 处方表

字段名	数据类型	约束	描述
pres_id	bigint	主键	处方 ID
registration_id	bigint	外键	挂号 ID
user_id	bigint	外键	开方医生 ID
total_amount	decimal	非空	处方总费用
pres_name	varchar	非空	处方名
medical_advice	varchar	非空	嘱托
therapy	varchar	非空	使用事项
fetch_status	int	非空	领取状态
pay_status	int	非空	付款状态
finished_status	int	非空	完成状态

6) 用户角色表。此表记录了用户与对应角色的关系, 包括用户 id、角色 id。表结构如下表 4-6 所示。

表 4-6 用户角色表

字段名	数据类型	约束	描述
U_R_Id	bigint	主键	
user_id	bigint	外键	用户 ID
role_id	bigint	外键	角色 ID

7) 角色信息表。此表记录了角色的详情, 包括角色 id、角色名称等信息, 表结构如下表 4-7 所示。

表 4-7 角色信息表

字段名	数据类型	约束	描述
role_id	bigint	主键	角色 ID
role_name	varchar	非空	角色名称
avaliable	int	非空	是否可用

8) 账单表: 账单记录了此次付款缴费的项目详情, 包括账单号、挂号 ID、包含项 ID、包含项目类型等信息。表结构如下表 4-8 所示。

表 4-8 账单表

字段名	数据类型	约束	描述
bill_id	bigint	主键	账单 ID
registration_id	bigint	外键	挂号 ID
record_id	string	非空	包含项 ID
type	int	非空	包含项目类型
total_amount	decimal	非空	总金额
bill_pay_status	int	非空	付款状态

4.4 本章小结

本章主要阐述系统的设计方案, 以划分的功能模块为蓝本, 完成系统从整体到局部分层的设计。在整体设计上, 从架构选型及前后端数据交互方式进行概述, 明确所选用的技术, 在局部分层中, 划分出各层的交互关系及发挥的职责。然后针对每个功能模块, 结合用例图及时序图对模块内的功能进行详细说明。最后针对数据库进行设计, 对涉及的重要表的字段进行阐述。

5 系统实现与测试

本章主要是对系统设计部分进行编码实现，先以 Web 页面效果图给以直观的功能展示，对其核心功能使用简洁的代码辅以阐述，且结合现有的资源，对系统进行测试。系统功能模块的实现分为如下六个模块：用户信息、医生排班、预约挂号、门诊医生、门诊收费、门诊药房，下面分别介绍系统实现及测试部分。

5.1 系统开发环境

本系统使用 IntelliJ IDEA 2019 作为集成开发工具，配置 JDK 1.8 的 Java 运行环境；采用 Spring Boot 2.4.1 + MybatisPlus 3.4.1 的系统框架进行开发构建 Web 服务；采用开源的 Mysql 5.7 关系型数据库，辅以数据库管理工具 Navicat 来设计和管理数据库表。借助 Maven 包管理工具对项目依赖包进行管理；使用 Git 版本控制工具进行多人协助开发，Postman 作为接口测试工具；采用 Jenkins 自动化构建部署工具，对项目进行自动打包，部署在 Linux 服务器上提供 Web 服务。使用 Nginx 进行服务器的反向代理和负载均衡。

5.2 系统实现

5.2.1 用户信息管理模块的实现

用户信息管理模块主要是对用户访问控制及用户一些基本信息的管理，访问控制包括用户登录的流程控制及用户访问资源链接时的权限控制。维护用户信息主要包括查看、修改、删除、添加用户信息操作。

访问控制，主要功能是维护系统的用户和角色，本模块只有管理员才有权限进行操作，本系统使用 Shiro 权限框架做登录和权限校验的处理。当用户访问系统资源时，请求会先经过身份认证过滤器，过滤器拦截获取访问 URL，然后将 URL 与 Shiro 配置类的过滤器链从上往下进行比对，如果命中则使用该过滤器进行权限验证。配置类的过滤器链如下图 5-1 代码所示。

```
@Bean
public ShiroFilterFactoryBean shiroFilter(SecurityManager securityManager) {
    Map<String, String> filterChainDefinitionMap = new LinkedHashMap<>(); //过滤器链
    //匿名可以访问，也就是游客模式
    filterChainDefinitionMap.put("/pub/**", "anon");
    //登录用户才可以访问
    filterChainDefinitionMap.put("/authc/**", "authc");
    //管理员角色才可以访问
    filterChainDefinitionMap.put("/admin/**", "roles[admin]");
    //有发药权限才可以访问
    filterChainDefinitionMap.put("/drugStore/release_drug", "perms[drug_release]");
}
```

图 5-1 过滤器链

改配置表明，访问 pub 公共模块下的，不需要登录即可以访问；访问 admin 管理员模块下的接口，需要有管理员角色；访问药品模块的发药接口，需要有药品发放权限。

当命中过滤器的规则后，需要取出用户的权限信息，此时需要自定义一个 Realm 来重写 AuthorizingRealm 的授权方法，在授权方法中得到用户的角色及权限信息。最后交给 Shiro 的权限校验实现类进行校验即可。授权代码如下图 5-2 所示。

```
protected AuthorizationInfo doGetAuthorizationInfo(PrincipalCollection principals) {
    User newUser = (User) principals.getPrimaryPrincipal();
    //查数据库中关于此用户的角色及权限信息
    User user = userService.findAllUserInfoByUsername(newUser.getUsername());
    for (Role r : user.getRoleList()) { //遍历角色列表
        RoleList.add(r.getName()); //添加角色
        for (Permission p : r.getPermissionList()) { //遍历权限列表
            PermissionList.add(p.getName()); //添加权限
        }
    }
}
```

图 5-2 用户授权代码

在前端登录页面，输入登录相关信息，比如账户名、账户密码，通过前端的校验后，就会进入后端的登录业务逻辑行验证。验证通过后将使用 JWT 工具产生一个身份 Token，并且将此 Token 放入 Redis 缓存。登录界面如下图 5-3 所示

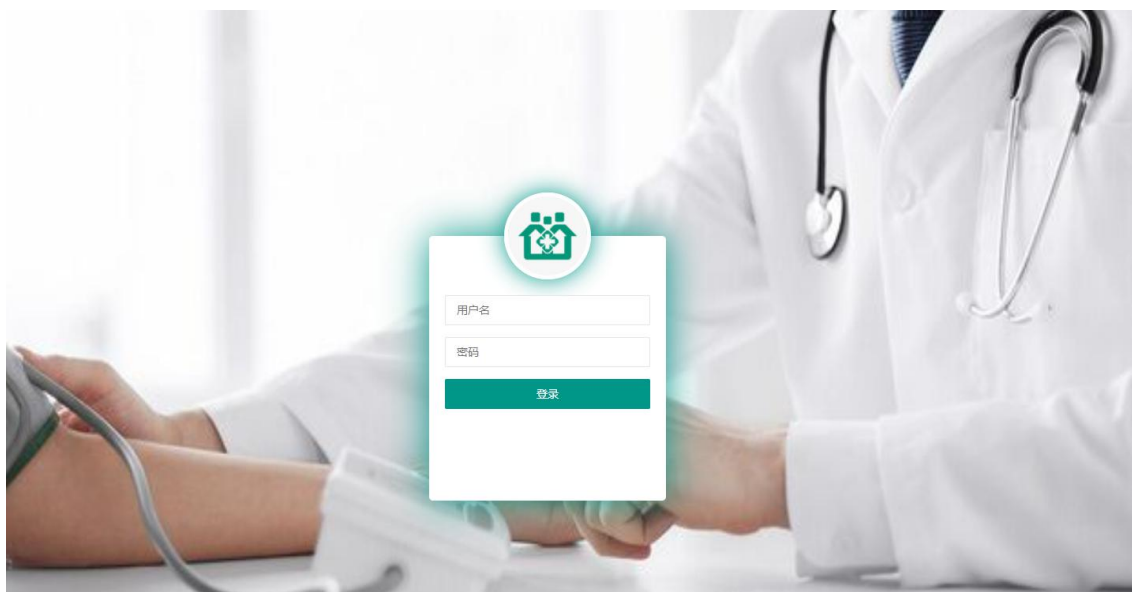


图 5-3 用户登录界面

管理员登入系统后, 点击用户管理菜单, 即可进入用户管理首页。点击用户管理菜单时, 发送请求到用户管理查询接口, 该接口读取数据库的用户表, 获取第一页的数据, 包括用户信息及角色描述信息, 然后数据以 JSON 格式进行返回, 前端以分页列表的形式进行展示。用户管理界面如下图 5-4 所示。

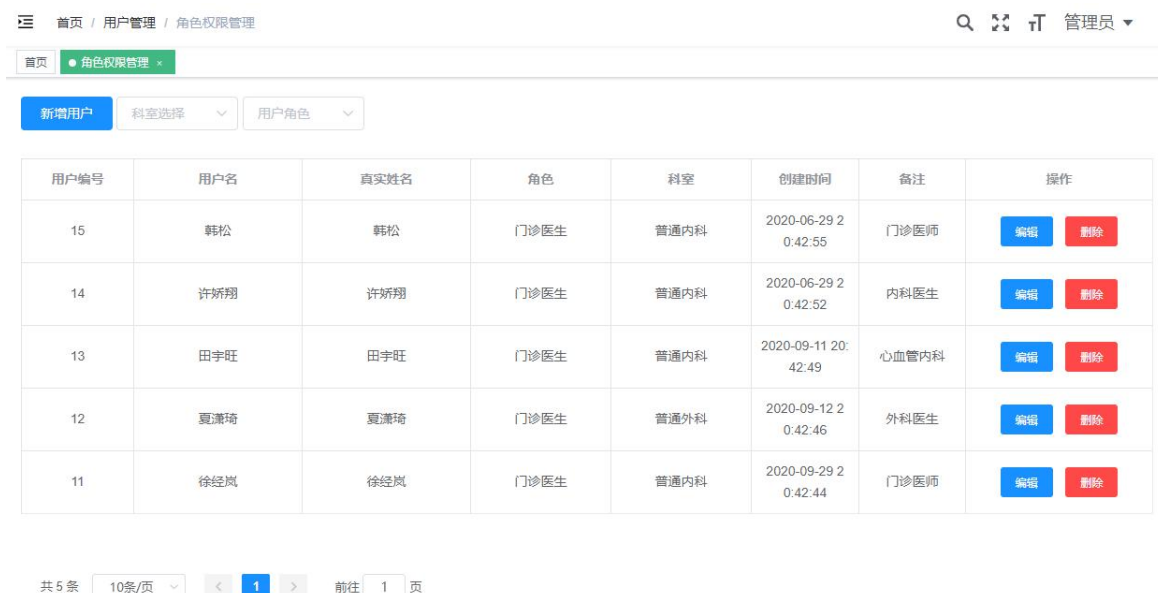


图 5-4 用户管理界面

在界面的最左上方是一个新增按钮, 提供了新增用户的功能。新增功能的后面是搜索功能, 提供了用户多组合条件查询, 如单独根据科室、用户角色或者科室及

用户角色进行人员查询。在列表中显示了用户的一些基础信息，如用户姓名、真实姓名、角色、所属科室、用户角色描述等。在每一个用户后面都有一列操作标签，第一个是编辑标签，点击后，在弹出的页面可以对用户的信息进行修改，包括对基本信息及角色、科室的修改。编辑标签后面是删除按钮，点击删除按钮，在弹出的提示页面选择删除此用户后，会将此选择的用户的使用状态改为不可用，实现逻辑上的删除。点击编辑标签进入用户信息修改界面，界面如下图 5-5 所示。

图 5-5 分配角色页面

在上图修改用户信息界面中，可以修改用户的角色信息，在进入这个界面之前，会调用 Redis 缓存服务的查询角色接口拿到系统所有的角色信息。该系统中设置了超级管理员、门诊医生、药房医生、收费员等七种角色。每种角色拥有不同的权限，超级管理员角色，这种角色拥有所有的菜单权限，是系统的权限最高者，门诊医生角色只拥有门诊医生菜单的访问权限。选中其中需要分配的角色及科室后，点击确认按钮，即可调用用户信息模块的修改用户信息接口，将用户的写入到用户角色表

及用户表中。

5.2.2 医生排班管理模块的实现

医生排班管理模块主要的功能是维护医生坐诊上班的时间，有新增、查看、修改医生排班计划的操作，只有管理员有权限进行操作。

排班管理员需要维护医生的日常坐诊信息，所以需在排班页面上进行排班管理，排班规则页面效果图如下图 5-6 所示。

Figure 5-6 shows the 'Doctor Scheduling Rule Management' page. At the top, there's a breadcrumb trail: '首页 / 医生排班管理 / 医生排班管理'. Below it, a navigation bar contains '首页', '排班表', and '医生排班管理' (the last one is active). The main content area has a '科室选择:' dropdown set to '神经外科', with '查询' and '清空' buttons. Below this is a table with columns: '医生名称', '星期一 上午', '星期一下午', '星期二 上午', '星期二 下午', '星期三 上午', '星期三 下午', '星期四 上午', '星期四 下午', '星期五 上午', '星期五 下午', '星期六 上午', '星期六 下午', '星期日 上午', '星期日下午', and '排班限额'. The table lists three doctors: Ma Ke (with limits 40, 30, 20), Ran Bing, and Zhang San. At the bottom, there are input fields for '规则名称' (set to '神经外科排班') and '规则描述' (set to '9.28-10.7'), along with '取消' and '保存修改' buttons.

图 5-6 展示排班规则页面

在排班列表展示中，显示了神经内科的排班医生的信息，包括医生的名称、具体时间及排班限额。在列表的上方是一个科室选择下拉框，选择好科室后点击查询按钮，就可以看到科室医生的排班情况。此时如果需要修改，通过在单元格中点击排班时间，对以前的时间进行改动，然后在规则名称输入框给当前修改的排班规则设置一个清晰易懂的名字及描述，然后点击生成排班计划，前端页面将排班规则转换为 Json 格式的数据，以 Post 的方式通过 Http 协议发送后台排班规则模块的修改排班规则接口（updateRule），接口调用其服务类的方法，将传输的数据写入到数据库的排班规则表和排版规则项表中。修改排班规则，具体代码逻辑如下如 5-7 所示。


```
public CommonResult updateRule(@PathVariable Long id, @RequestBody SmsSkdRuleParam smsSkdRuleParam) {  
    //删除排班计划  
    int skdRuleCount = skdService.deleteSkdById(id);  
    //删除排班计划中的排班计划项  
    int skdRuleItemCount = skdService.deleteSkdItem(smsSkdRuleParam.getSmsSkdRuleItemParamList())  
    //插入到排班计划  
    SmsSkdRule smsSkdRule = new SmsSkdRule();  
    BeanUtils.copyProperties(smsSkdRuleParam, smsSkdRule); //设置属性  
    skdService.createSkd(smsSkdRule); //插入数据到排班计划表中  
    //遍历排班项并全部插入  
    skdService.createSkdItem(smsSkdRuleParam.getSmsSkdRuleItemParamList())  
    int count = skdRuleCount + skdRuleItemCount;  
    if (count > 0) {  
        return CommonResult.success(count, "更新成功");  
    }  
    return CommonResult.failed("更新失败");  
}
```

图 5-7 修改排班规则的业务逻辑代码

该方法采用的逻辑是先删除原先存在的排班计划，然后新增现在提交的排班计划。该操作先根据参数中的排班计划 ID，调用 SkdService 的 deleteSkdById 方法删除排班计划及排班计划中的排班计划项中的数据。然后调用 SkdService 的 createSkd 方法新增排班计划表及排班计划中的排班计划项中的数据。

5.2.3 预约挂号管理模块的实现

挂号是来院就诊的第一个环节，完成挂号后才能继续后序的就诊、缴费、检查、拿药等环节。该模块主要的功能分别为：查看每个科室医生坐诊详情及选取医生进行挂号。

1) 查看科室医生坐诊详情。此功能的实现分为两个部分，一个部分是根据科室查询医生的排班信息。第二部分是排班模块的医生信息不够完善，未包括医生的详细信息，比如专长介绍，坐诊级别等，此时调用用户管理模块的根据医生 ID 查询医生详情。当患者挂号选择外科医生排班列表时，会显示该科室的医生坐诊情况，坐诊详情图片如下图 5-8 所示。

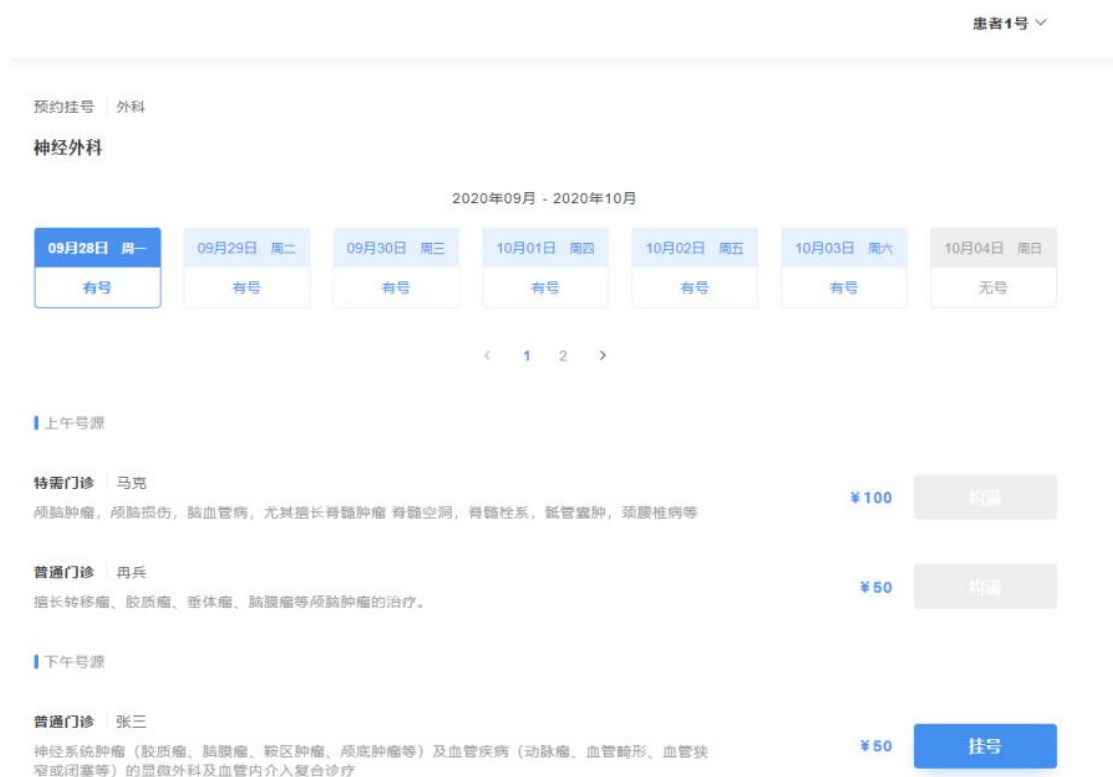


图 5-8 患者预约挂号界面

从上图可以看到是外科医生的坐诊信息，最上面是时间显示栏及当天是否有号，下部分显示的是坐诊医生的详细信息，比如坐诊时间、挂号类型、医生专长、挂号金额及今日时候是否还有号等详细信息。

此模块涉及的主要方法为根据科室名称查询排班计划及医生详情 `getRuleAndDocInfoByDepName`。具体代码逻辑如下如 5-9 所示。

```
public Result<?> getRuleAndDocInfoByDepName(String depName, DateTime time) {
    List<DoctorResult> list = new ArrayList<>();
    SkdRule skdRule = schService.getRuleByDepName(depName); //根据部门名称得到医生排班
    List<SkdRuleItem> SkdRuleItemList = skdRule.getSkdRuleItemList(); //得到排班项中医生的信息
    for (SkdRuleItem skdRuleItem : SkdRuleItemList) { //遍历每个排班项对象
        Long docId = skdRuleItem.getId(); //医生ID
        String daysOfWeek = skdRuleItem.getDaysOfWeek(); //工作日
        //是否是今天排班，这里做简单处理，只要星期是一样的就排班
        if (daysOfWeek.equals(time.getDayOfWeek())) {
            DoctorResult doctorResult = new DoctorResult();
            Doctor doctor = UInfoService.getDoctorInfoById(docId); //得到医生详细信息
            //设置医生上班属性，午别、
            doctorResult.setNoon(skdRuleItem.getNoon()); //设置为上午还是下午
            ...//设置疾病专长、挂号级别、挂号费用
            doctorResult.setRemain(getRemainByDoctorId(docId)); //设置医生剩余挂号数
            list.add(doctorResult);
        }
    }
}
```

图 5-9 查看科室医生坐诊详情的具体实现代码

此方法的调用逻辑是先调用 `getRuleByDepName` 函数返回医生排班计划对象，

该对象包含一个医生排班项对象列表。医生排班项对象里面存储了排班医生的具体信息，包括医生 ID、排班时间、挂号限额、剩余挂号余额等信息。遍历这个医生排班项列表，根据医生 ID 调用 `getDoctorInfoByID` 得到医生详细信息如专长、挂号级别、挂号费用等关键信息，对排班计划中的医生信息属性进行填充。向患者展示医生的排班信息时，以一天一天的形式展示比较符合实际情况，最后将工作时间相同的医生信息放在一个集合对象里面返回即可完成查看科室医生坐诊详情的操作。

2) 挂号。当患者选好了医生后，如果此医生还有挂号名额，那就点击挂号按钮，即可完成挂号。患者挂完号以后在门诊挂号收费平台即可看到患者的挂号信息，门诊挂号收费台的页面图如下图 5-10 所示。

操作	状态	就诊号	姓名	性别	出生日期	看诊日期	看诊科目
缴费 退费	待收费	2020060618172312	黄强辉	女	2000-11-01	2020-09-28	外科
缴费 退费	待收费	2020060618172310	马连良	男	2000-11-01	2020-09-28	神经内科
无	待缴费	2020060618175209	赵兰	女	2003-10-29	2020-09-27	神经内科
退号 补打	未缴费	2020101018171000	李虎	女	1990-06-28	2020-09-28	心血管内科

图 5-10 患者挂号信息

上图显示了门诊挂号患者的信息，比如患者挂号记录的状态，比如是否缴费、医生诊断状态；患者的就诊号，此就诊号可以唯一对应一位患者；患者的姓名、性别；挂号科室，如外科、神经内科、心血管内科等。

挂号操作涉及到用户信息模块的患者信息的查询、排班服务模块的医生挂号名额减一，挂号预约模块的保存挂号记录，账单服务模块的新增账单接口。涉及的接口为 `createRegistration` 创建挂号记录，具体代码逻辑如下如 5-11 所示。

```
@RequestMapping(value = "/registration", method = RequestMethod.POST)
public CommonResult createRegistration(Long medNo, Doctor doctor) {
    Patient pat = UInfoService.getPatInfoByMedNo(medNo); //得到患者详细信息
    //此挂号医生的挂号人数减一
    Boolean b = skdService.decDocRegNumberByOne(doctor.getId)
    if(b) {
        Registration reg = new Registration(); //构建患者挂号对象
        //设置挂号对象的属性: 患者id, 挂号医生id, 就诊时间, 就诊状态, 付款状态等
        reg.setPatientId(pat.getId);
        ...
    }
    Long regId = regisService.saveRegistration(reg); //保存挂号对象
    //设置账单的属性: 挂号id, 类型, 总金额, 创建时间、付款状态等
    BillRecord billRecord = new BillRecord();
    billRecord.setRegistrationId(regId); //设置挂号id
    ...
    feeService.saveBillRecord(billRecord); //保存账单记录
}
```

图 5-11 挂号对象的逻辑代码

此方法先用参数里面患者的就诊号,调用 `getPatientByMedNo` 得到患者的信息详情。构建一个患者挂号记录对象,设置挂号对象的相关属性如时间、患者 id、挂号医生的 id、等信息,保存挂号记录前还需要将挂号医生的剩余挂号余额调用 `decDocRemainByOne` 方法减一操作。如果剩余挂号余额减成功,调用 `saveRegistration` 方法保存患者的挂号记录。最后构造账单对象,设置账单对象的挂号 id、挂号记录 id、类型(0、挂号收费)、总金额等信息,付款状态用于后面收费模块进行判断,最后调用 `saveBillRecord` 方法将构建的账单记录对象保存即可完成挂号的流程。

3) 短信就医提醒服务。本模块定时任务使用 `Cron(0 0 7 * *)`表达式来进行,该表达式表上每天早上 7 点的时候会执行查询挂号列表的方法,该方法调用挂号模块的查询今日挂号记录 API 得到今日挂号记录的信息。然后对其进行分装,分装其属性,比如手机号、挂诊科室、患者名称等。然后将此记录发送到 Kafka 消息队列的挂号就医提醒 `his.registration.ttl` 主题中,这个主题的消息会被监听这一主题的消费者给消费,然后判断主题的消息是否已经到达了预定(`delayTime`)的时间,如果是则将其放入 `his.registration` 消息主题中,由监听该主题的消费者进行消费,然后调用阿里云短信服务的 `send` 方法即可发送就医提醒短信。`send` 具体代码逻辑如下如 5-12 所示。


```
private boolean send(String phone, Map<String, Object> param) {  
    //整合阿里云短信服务, 设置相关参数  
    DefaultProfile profile = DefaultProfile.  
        getProfile(ConstantPropertiesUtils.REGION_ID, //区域ID  
            ConstantPropertiesUtils.ACCESS_KEY_ID, //阿里云控制台设置的访问keyID  
            ConstantPropertiesUtils.SECRET); //阿里云控制台设置的访问密钥  
    IAcsClient client = new DefaultAcsClient(profile);  
    CommonRequest request = new CommonRequest();  
    //设置请求参数属性, 发送方式、产品域名、产品版本  
    request.setMethod(MethodType.POST);  
    ...  
    request.putQueryParameter("PhoneNumbers", phone); //接收消息的手机号  
    request.putQueryParameter("SignName", "东方医院"); //签名名称  
    request.putQueryParameter("TemplateCode", "SMS_180051135"); //模板code  
    request.putQueryParameter("TemplateParam", JSONObject.toJSONString(param)); //填充模板的参数  
    //调用方法进行短信发送  
    CommonResponse response = client.getCommonResponse(request);  
}
```

图 5-12 发短信逻辑代码

此方法中需要注意的是要接收的短信的手机号；签名名称，该签名名称必须是阿里平台通过了的合法名称，这里设置为东方 xx 医院；设置发送消息的模板，此模板为自己在阿里云平台设置的模板 Code，这里的模板设置为：亲爱的\${name},您于今日\${date}\${noon}有预约挂号信息,挂号科室为\${deptname},请提前出发看诊。如果您不是此人，请忽略这条消息。如果此时传入的参数里面的 name 属性为张三，时间为 2020-09-28，noon 为下午，科室名称为神经内科。消息发送成功后对应的患者收到的消息为：【东方 xx 医院】亲爱的张三，您于 2020-09-28 下午有预约挂号信息，挂号科室为神经内科，请提前出发看诊。如果您不是此人，请忽略这条消息。

5.2.4 门诊医生工作台模块的实现

门诊医生工作台是对病人看病及诊断的主要场所，也是整个系统的重点。其主要是对患者的病情信息进行诊断、开立检查项、录入就诊信息、开处方的操作。该模块的业务逻辑相对比较复杂，本模块主要说明对患者的病情进行诊断、检查模块、开处方、处方逾期未交费处理四个模块的具体实现。

1) 门诊诊断。该模块涉及两部分，一部分是当医生进入门诊医生工作台页面时，需要在首页展示自己的患者列表。第二部分是根据患者名字或者就诊号查询患者的病例，包括初诊、复诊及诊毕，及医生对患者的病例进行增加、修改的操作。

医生登录系统，点击左侧功能栏的门诊医生工作台，进入门诊医生工作台页面，页面效果如下图 5-13 所示。

The interface shows a sidebar on the left with three tabs: '本人' (Myself), '科室' (Department), and '待诊患者' (Waiting Patients). The '待诊患者' tab is active, displaying a table of waiting patients with columns for '就诊号' (Visit Number), '名字' (Name), and '年龄' (Age). Below this is a '诊中患者' (In Progress Patients) section with a similar table. At the bottom is an '已诊患者' (Completed Patients) section with another table.

The top navigation bar includes a '当前病人' (Current Patient) section with fields for '姓名' (Name), '就诊号' (Visit Number), '性别' (Gender), and '年龄' (Age). Below this is a row of tabs: '病历首页' (Medical History Home), '检查申请' (Check Request), '检验申请' (Test Request), '门诊确诊' (Outpatient Confirmation), '成药处方' (Medicine Prescription), '草药处方' (Herb Prescription), '处置申请' (Treatment Request), and '患者账单' (Patient Bill).

The main content area contains a '病史内容' (Medical History Content) section with a '主诉' (Chief Complaint) field and a '现病史' (Current History) field. Below these are fields for '既往史' (Past History), '过敏史' (Allergy History), and '体格检查' (Physical Examination). A '发病时间' (Onset Time) field is also present. At the bottom is a '评估诊断' (Evaluation and Diagnosis) section with a table for 'ICD编码' (ICD Code), '名称' (Name), and '编码' (Code). A '提交初诊' (Submit Initial Visit) button is located at the bottom right.

图 5-13 门诊医生工作台界面图

上图左侧是展示的是医生的诊断患者的信息，包括已缴费待诊、诊中及已诊患者，包括患者的就诊号、名字及患者年龄信息，方便医生根据就诊号查询患者病历信息。此模块涉及的接口为 `listPatientByDocId` 通过医生 ID 得到患者列表，具体代码逻辑如下如 5-14 所示。

```
@RequestMapping(value = "/listPatientByDocId", method = RequestMethod.POST)
public Result<?> listPatientByDocId(Long DoctorId) {
    //根据医生id得到挂号列表
    List<Registration> registrationList = regisService.selectRegistrationByDocId(DoctorId);
    for (Registration registration : registrationList) {
        DiagnosisPatientResult diagnosisPatientResult = new DiagnosisPatientResult();
        //根据患者id得到患者具体信息
        Patient patient = UInfoService.seclctPatientByPatId(registration.getPatientId());
        //设置患者详细信息 名称、患者地址、性别等
        diagnosisPatientResult.setPatientName(patient.getName()); //设置患者名称
        if (registration.getStatus() == 1) { //1、待诊
            personalWaitList.add(diagnosisPatientResult); //加入到个人待诊列表
        } else if (registration.getStatus() == 2) { //2、诊中
            personalDuringList.add(diagnosisPatientResult); //加入到个人诊中列表
        } else if (registration.getStatus() == 3) { //3、诊毕
            personalEndList.add(diagnosisPatientResult); //加入到个人已诊列表
        }
    }
}
```

图 5-14 患者列表具体实现

通过医生 ID 调用注册服务的 `selectRegistrationByDocId` 方法得到与医生相关的挂号列表。遍历每一个挂号列表，通过挂号对象的就诊状态 `status` (1、待诊，2、

诊中，3、已诊）对患者进行分类。为了得到患者的基本信息，通过挂号对象的患者ID 属性调用用户信息服务的 selectPatientByPatId 得到患者的基本信息，最后将这些属性进行封装以 JSON 的数据形式返回填充页面即可。

2) 在医生工作台页面的左上方是患者搜索模块，根据患者名称查询患者，在输入框输入患者名称点击查询按钮或者直接点击左边的用户栏，就会调用查询患者病历信息 API 将患者的基本信息及诊断信息填充至图的右侧。如果患者是初诊，则患者的诊断病历首页就是为空，如果是复诊患者就显示最近的一条就诊记录。如图 5-15 所示显示的是复诊患者的最新病历详情。

隐藏患者栏 患者姓名: 卢照邻 病历号: 600606 年龄: 33 性别: 男 诊毕

< 病历首页 检查申请 检验申请* 门诊确诊* 处置申请* 成药处方 草药处方* 费用查询*

回暂存 提交 清空所有 刷新

病史内容:

主诉: 右下腹痛伴恶心。

现病史: 该患于8小时前无诱因出现腹部疼痛，初表现为上腹部隐痛，4小时后疼痛逐渐加重并转移至右下腹固定，无腰部及会阴部放射痛，呈阵发性发作，伴有恶心、呕吐数次，呕吐物为胃内容物，量共约200毫升，未经任何诊治，今因腹痛再次就诊。

现病治疗情况: 该患以转移性右下腹痛伴恶心、呕吐8小时于2008年06月11日入院。

既往史: 无结核及肝炎病史，无糖尿病及心脏病、高血压病史，无药物过敏史及手术史。

过敏史: 无

体格检查: 无

评估/诊断:

西医诊断

ICD编码	名称	发病时间
☐ A00.152	埃尔特霍乱	2019-03-20 15:21

中医诊断

ICD编码	名称	发病时间
暂无数据		

检查建议: 血常规

注意事项: 无

提交

图 5-15 患者的病历记录详情

该病例分为两部分，上一部分是包括主诉、既往史、先病史、过敏史、这些基

本信息，医生根据患者的就医进度对患者的病情病历逐步的填充，填写完成点击提交按钮即可调用病例模块的更新接口保存数据即可。下半部分是医生根据行医经验及检查结果报告对患者添加的病情诊断，即该症状是属于什么疾病导致的，医生新增诊断后，点击提交按钮即可将诊断信息以 Post 的方式通过 Http 协议发送到病历管理模块的病历更新 API，将数据写入到数据库的病历表中即可完成此次的诊断。

3) 检查模块。此模块是无法对患者病情进行确诊，通过开立检查项目来进行服务确诊。开立检查项目的页面图如下图 5-16 所示。

当前病人: 姓名: 赵兰 就诊号: 2020060618175209 性别: 女 年龄: 16岁

病历首页 检查申请 检验申请 门诊确诊 成药处方 草药处方 处置申请 患者账单

①新增项目 ②删除项目 ③开立项目 ④作废项目 ⑤暂存 ⑥取出暂存项 ⑦刷新

项目金额总计: 190.00 元

项目编码	项目名称	执行科室	单位	单价	状态	
☐ 210101001	普通透视	神经内科	每个部位	5	已执行	查看结果
☐ 210103001	气脑造影	放射科	次	80	已作废	
☐ 210101003	床旁透视与术中透视	放射科	半小时	40	已作废	
☐ 210103001	气脑造影	放射科	次	80	已登记	
☒ 210101003	床旁透视与术中透视	放射科	半小时	40	未开立	检查要求
☒ 210101004	C型臂术中透视	放射科	半小时	150	未开立	检查要求

图 5-16 开立检查项目页面

该页面的中间位置是新增、删除、开立、作废项目按钮。新增项目是作为开立检查项目的开始，点击后，在页面点击新增项目，即可调用缓存服务的查询检查项目 API 得到所有的检查项目，选中相应的项目进行添加，然后在每个检查项目后面有一个检查要求标签，点击这个标签，输入对应的项目信息如此次检查的目的、要求、检查部位等信息保存即可，最后添加完检查项目后点击开立项目调用检查服务的开立项目接口即可。添加检查项目的页面如下图 5-17 所示。

项目名称	项目编码	价格
脑室碘水造影	210103003	60
气脑造影	210103001	80
C型臂术中透视	210101004	150
床旁透视与术中透视	210101003	40
食管钡餐透视	210101002	15
普通透视	210101001	5

图 5-17 检查项目列表

开立完检查项目后，患者在指定的时间缴费，且在相应的部门登记做完做完检查后，医技医生即可看到患者的开立项目。患者检查后，门诊医技人员在页面上填写检查结果及影像图片，然后调用检查模块的检查结果上传 API 保存此次的检查结果。医技工作平台的页面图所下图 5-18 示。

病历号	姓名	年龄	性别	开立项目	检查部位	状态	操作
1	赵兰	16岁	女	气脑造影	头	已登记	上传结果
4	高大山	20岁	女	C型臂术中透视		未登记	登记 林录

图 5-18 门诊医技工作台页面图

在界面中间显示的是患者的检查项目信息，包括患者的姓名、逻辑、开立项目、检查部位、还有可开立的状态。检查完了以后点击后面的上传结果按钮，在上传结果页面需要填写此字段信息，及上传检查时的图片，填写完成调用检查模块的上传检查结果接口（/check/uploadResult）即可。具体代码逻辑如下如 5-19 所示。

```
@PostMapping("uploadResult")
public Result fileUpload(MultipartFile file, @Valid checkResult checkResult) {
    MechanicItemRecord mecItemRecord = new MechanicItemRecord(); //检查记录对象
    //设置上传文件的属性, 比如检查结果
    BeanUtils.copyProperties(checkResult, mecItemRecord);
    //上传到阿里oss的图片地址
    String url = fileService.upload(file);
    checkService.update(mecItemRecord);
}
```

图 5-19 上传检查结果代码

该上传文件通过 post 方式发送 HTTP 请求到/check/uploadResult 接口, 携带的文件以 MultipartFile 形式进行接收, 结构化数据以类对象参数形式接收。文件上传服务调用 upload 上传到阿里云中后返回的是阿里云的存储路径, 然后将检查结果及返回路径调用检查服务保存检查结果 API 保存到数据库即可。数据库中的数据, 如下图所示, 其中的 result_img_url_list 即为阿里云的存储地址。

status	aim	demand	log_status	check_result	result_img_url_list
4	目标	需求	1	两侧脑内未见多	http://zain-images.oss-cn-beijing.aliyuncs.com/mall/images/20190705/1012111944f5d53269d9d99e06.jpg
3	辅助确诊	检查感部分	1	头疼	http://zain-images.oss-cn-beijing.aliyuncs.com/mall/images/20190705/1012111944f5d53269d9d99e06.jpg
4	辅助确诊	检查感部分	1	胃疼	9e06.jpg,http://zain-images.oss-cn-beijing.aliyuncs.com/mall/images/20190705/201304260934009696.jpg

图 5-20 检查数据库记录

在患者再次进行复诊的时候, 医生可以看到自己给患者开过的检查检验的结果, 此结果是通过调用检查服务的查询患者检查结果 API 拿到的结果, 调用此 API 需要传递患者挂号 ID 及检查项目 ID。在图 xx 所示的已经开立的检查项目气恼造影项目的最尾方有一个查看检查结果标签, 点击此标签进入检查结果界面就可以查看技师医生上传的检查结果及检查图片, 图片如下图 5-21 所示。

在界面左上方是门诊开立的处方名称及状态，右上方是增方、删方、开立、作废、增药、删药的功能。增方是此模块的最开始按钮，点击增方按钮，会要求医生输入处方名称。如果不输入处方名称，默认名为“新增处方+数字”。处方名字确定后，点击“增药”，出现药品信息页面，在页面最上方输入药品编码、拼音码后，点击查询按钮，调用药品信息管理模块的模糊查询 API 查询药品信息，然后选择药品后，医生需要填写药品用法、用量、频次、数量等信息，在填写完药品信息后，点击“添加”完成该增药操作。添加完所有的药品后，点击开立按钮，直接调用处方管理模块的开立处方 API 保存处方数据。开立处方接口业务逻辑代码如下图 5-23 所示。

```
@RequestMapping(value = "/apply_prescription", method = RequestMethod.POST)
public CommonResult applyPrescription(@RequestBody MedicinePrescriptionRecordParam medPreParam) {
    //构建成药处方对象
    MedicinePrescriptionRecord medPreRecord = new MedicinePrescriptionRecord()
    //设置成药处方对象状态：总金额、处方成药项对象、患者挂号id等
    BeanUtils.copyProperties(medPreParam, medPreRecord);
    medPreRecord.setFetchStatus(2); //未领药
    medPreRecord.setPayStatus(0); //未付款
    medPreRecord.FinishedStatus(1); //可用
    presService.SaveMedicinePreRecord(medPreRecord) //保存
    //设置账单的属性：挂号id，类型，总金额，创建时间、付款状态等
    BillRecord billRecord = new BillRecord();
    billRecord.setRegistrationId(regId); //设置挂号id
    ...
    feeService.saveBillRecord(billRecord); //保存账单记录
}
```

图 5-23 开立处方业务逻辑代码

在调用开药模块前，医生需要先进行添加药品的操作，医生首先需要根据药品名称或者编号进行药品模糊查询，此时就需要调用药房管理模块的根据药品条件查询 API 拿到药品信息，根据得到的药品对象构建处方成药项对象。然后构建成药处方对象，设置其属性如总金额、处方成药项对象、患者挂号 id、医生 id 等信息，调用存储接口 SaveMedicinePreRecord 将开方内容存入数据库中。此时需要构造账单对象，设置账单对象的挂号 id、付款状态（0、未付款）等信息，付款状态用于后面收费模块进行判断，最后调用 SaveBillRecord 方法将账单对象保存即可完成处方开立的流程。

4) 逾期处理。当完成处方开方以后，处方一段时间内是有效的，患者在这段时间内不缴费的话，需要将处方从可用状态改变为已过期状态。设置为过期状态的处方，门诊收费部门将查询不到此处方信息，患者无法对其进行缴费。处方逾期处理实现代码如下图 5-24 所示。

```
@Component
public class KafkaPrescriptionReceiver {
    @Autowired
    private PrescriptionService presService;
    @Kafka(topics = "his.prescription")//处方消息主题
    public void handle(String msg) {
        String[] temp = msg.split("&");//解析得到处方id 与类型 （处方）
        MedicinePrescriptionRecord mp = presService.selectPreById(Long.parseLong(temp[0]));
        if (prec.getPayStatus == 0 && prec.getFinishStatus == 1) { //未缴费且为可用状态
            presService.unUsedPreById(Long.parseLong(temp[0])); //不改为不可用状态
        }
    }
}
```

图 5-24 处方超时未缴费代码

在处方开方完成后，将处方的信息传递到 Kafka 消息队列的处方开立延迟 his.prescription.ttl 主题中，这个主题里面的消息会被延迟一个 delayTime（一天）时间段后被消费。由对应的主题消费者消费，然后判断是否已经到达了一个 delayTime，如果是则将消息传递到真正处理处方的主题 his.prescription 中，由消费者消费且解析对应的消息，拿到处方 ID，调用处方管理模块的 selectPreById 拿到此处方信息，如果是未交费且可用状态的处方，说明此时已经逾期了，该改为过期不可用状态。调用处方管理模块的 unUsedPreId 方法将处方的状态改为不可用即可完成逾期处理的过程。

5.2.5 门诊收费管理模块的实现

门诊收费管理主要完成患者收费列表查询、自动核算收费价格、收费、打印收费凭据等功能。收费管理是其中重要的模块，涉及财产安全，所以需要在设计过程中需要充分考虑每一个具体细节的实现，确保整个模块能正常的对外提供服务。本模块主要说明患者收费列表查询及收费两个模块的具体实现逻辑。

1) 门诊收费员登录系统，点击左侧功能栏的收费标签，进入收费页面，然后根据患者的就诊号查询患者的待缴费信息，得到的患者缴费信息如下图 5-25 所示。



图 5-25 患者待收费列表界面图

在收费工作台的左侧栏显示的是患者的未收费及已收费项目，下面显示了患者的简要信息，比如患者就诊号，姓名。左侧上方的是一个搜索功能，通过在输入框输入患者名称或者直接点击患者列表，即可将患者的收费信息展示在页面的右侧表格里面。右侧的收费项显示了未收费的项目的详情，比如处方号、处方包含的药品名、开立的医生、价位、收费状态等信息。

该模块的业务逻辑相对比较复杂，涉及的对象比较多。涉及的接口为 `getPatientFeeListByMedNo` 通过患者就诊号得到患者收费列表。查询患者收费列表的业务逻辑代码如下图 5-26 所示。

```

public Result<?> getPatientFeeListByMedNo(String MedicalNo) {
    Patient patient = UInfoService.getPatientInfoByMedNo(MedicalNo); //根据就诊号得到患者信息
    //设置患者的基本信息
    patientFeeListResult.setPName(patient.getName()); //设置名称
    List<Registration> regs = regisService.getRegisByPatId(patient.getId()); //根据患者信息得到挂号列表
    for (Registration reg : regs) { //遍历患者挂号列表
        BillRecord billRecord = feeService.getBillRecByRegId(reg.getId()); //根据挂号id得到账单信息
        //1、未付款 账单付款状态
        if (billRecord.getStatus() == 0) {
            //1.1 药品收费 账单收费类型
            if (billRecord.getType() == 1) {
                BillRecordDetail billRecordDetail = new BillRecordDetail();
                //设置账单属性，总金额、付款状态、药品类型
                BeanUtils.copyProperties(billRecord, billRecordDetail);
                //获取处方项的详情
                MedicinePrescriptionRecord medPre = presService.getMedPresRecByPreId(billRecord.getRecordId());
                //设置包含的处方项详情、账单列表详情
                billRecordDetail.setMedicinePrescriptionRecord(medPre);
            }
            // 1.2 挂号收费
        } else if (billRecord.getType() == 0) {
            ...
        }
        //1.3 检查收费
        else if (billRecord.getType() == 2) {
            ...
        }
    }
    //2、已付款
    } else { //账单付款状态：已付款

```

图 5-26 患者收费列表逻辑实现代码

该接口先使用患者就诊卡号调用用户模块的 `getPatientInfoByMedNo` 得到患者信息对象。一个患者可能有多个挂号记录，调用挂号模块的 `getRegisByPatId` 方法得到患者挂号列表。然后遍历患者的挂号列表，调用 `getBillRecByRegId` 得到每一个挂号列表的账单记录。根据账单记录的付款状态（0、未付款，1、已付款）及收费类型（0、挂号收费，1、药品收费，2、检查收费）进行分类。由于向用户展示的时候需要展示账单包含的具体记录项详情，所以需要通过账单包含的记录项 `recordId` 属性得到此账单具体包含的项目详情。如果收费类型为 1，则此时的 `recordId` 则记录的是处方的 `id`。则需用通过调用处方模块的 `getMedPrescriptionRecByPreId` 得到成药处方记录，最后将这些信息分装返回即可。

2) 当门诊收费员勾选了收费项目，点击收费按钮进入结算界面，如下图 5-27 所示。

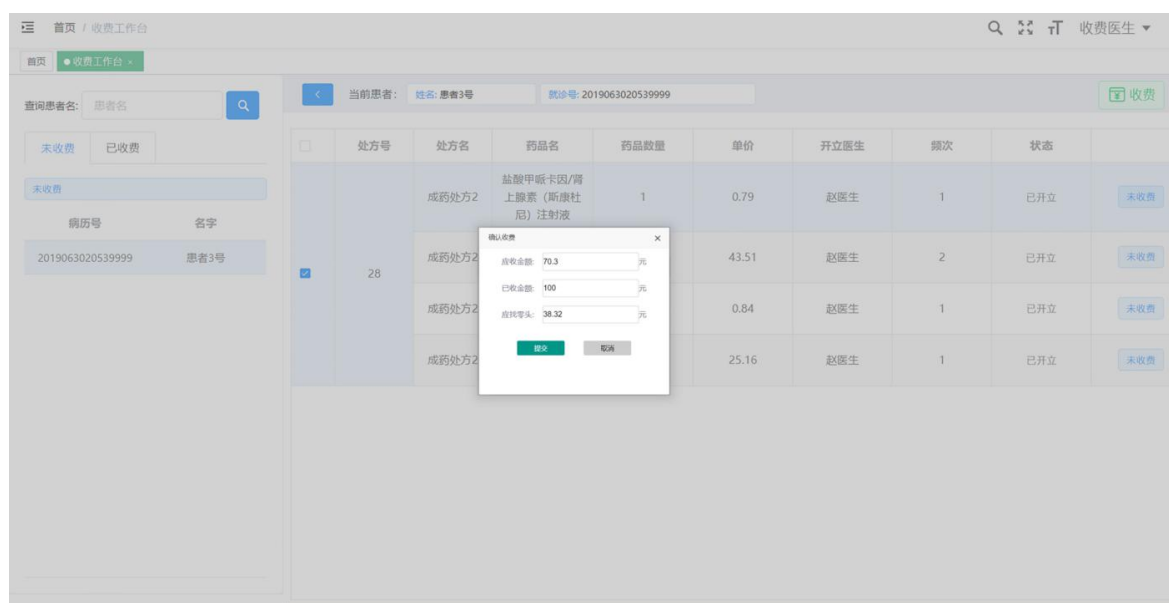


图 5-27 确认收费效果图

上图显示了点击勾选需要缴费的项目后点击缴费时，在弹出的确认缴费框里面显示了应收金额，这个是系统自己算出来的，不能修改。下面的一个是输入框，需要输入已收金额，输入后，系统自动结算出需要找的金额。

此模块的步骤相对简单，涉及的接口为 `charge`。收费代码的逻辑如下图 5-28 所示。

```
@PostMapping("/charge")
public Result<?> charge(ChargeParam chargeParam) {
    BillRecord billRecord = new BillRecord();
    billRecord.setId(chargeParam.getBillId()); //设置账单ID
    billRecord.setStatus(1); //设置为已收费
    boolean billFlag = feeService.updateBillRecByBillId(billRecord);
    if (billFlag) { //账单收费状态更改为已经收费
        InvoiceRecord invoiceRecord = new InvoiceRecord();
        //设置发票属性，收费人id、关联账单、实收金额、找回金额、时间等
        invoiceRecord.setChargePerId(chargeParam.getChargePerId()); //收费人id
        ...
        boolean invFlag = feeService.createInvRec(invoiceRecord); //保存发票信息
    }
}
```

图 5-28 收费代码逻辑

先根据账单 id 调用 `updateBillRecByBillId` 方法把账单及账单里面包含的记录项的付款状态改为 1（已付款）。如果更改成功，构建一个发票记录对象，设置其属性收费人 id，应收金额、实收金额、找回金额等必要信息。最后调用 `createInvRec` 将构建好的发票信息保存到数据库的发票表中即可。收费管理员然后在后台即可看到患者的缴费发票信息，发票信息如下如 5-29 所示。



图 5-29 门诊日结平台

在上图中的左侧栏中显示的是日结信息表的信息，包括收费员、创建时间、日结状态。在左侧栏的是一个时间范围查询，选择起始和截至日期，点击确定查询按钮，即可将这个时间段位的日结单及发票信息列出，上图右侧显示的就是上面点击确定缴费后自动生成的发票单据，包括发票号、发票金额、发票时间、支付类型等信息。

5.2.6 门诊药房管理模块的实现

门诊药房管理是门诊系统中的重要管理模块，主要完成药品信息维护、查询发药患者列表、发药的功能。本模块主要说明发药患者列表展示及发药功能的具体实现。

门诊发药。药品工作人员点击菜单栏的药房工作台标签后，进入药房工作台页面，页面效果如下 5-30 所示。



图 5-30 药房工作台界面

药房工作台界面是药房负责发药的人员的主要操作界面，页面左侧显示的是今日发药患者的信息，包括未发药和已发药患者列表，展示的是患者的基本信息，如病历号、患者姓名。左上方是一个搜索框，输入患者名字或者病历号进行模糊查询，调用门诊发药模块的查询患者发药信息接口，即可将患者的未发药信息及患者的基本信息填充在右侧的方框栏里面。勾选相应的发药处方后，点击右上角的发药按钮，将选中的处方药品信息调用药房工作台模块的患者发药接口，由其实现类将发药记录保存到数据库的药品处方记录表和药品项记录表中即可完成整个发药流程。

该流程涉及的接口有 listPatient、releaseDrug 接口，下面以 listPatient 展示发药患者列表的实现进行说明。具体代码逻辑如下如 5-31 所示。

```
public Result<?> listPatient(Date date, String medicalRecordNo) {
    //在处方表中根据时间查询
    List<MedicinePrescriptionRecord> recordList = presService.selectDrugPreRecord(date);
    //判断处方项状态为2(未领药)\3(已领药)分类，查询挂号Id并去重
    Map<Long, List<Long>> undoPatientToRecord = new Hashtable();
    for (MedicinePrescriptionRecord record : recordList) {
        if (record.getStatus() == 2) { //未领药
            ...//添加挂号id及处方记录列表
        } else if (record.getStatus() == 3) { //已领药
            ...//添加挂号id及处方记录列表
        }
    }
    //根据病人id再在患者表中封装病人信息
    for (Long key : undoPatientToRecord.keySet()) {
        //查挂号表
        Registration registration = regisService.selectRegistrationById(key);
        //查患者表，封装信息
        Patient patient = UInfoService.selectPatientById(registration.getPatientId());
        //传入患者病历号，则筛选患者
        if (medicalRecordNo != null) {
            ...//筛选患者
        }
        //设置返回结果的患者属性，患者id、处方id串，就诊号等信息
        result.setPatientId(patient.getId());
        undoPatientList.add(result);
    }
}
```

图 5-31 查询发药患者列表

该方法调用 selectDrugPreRecord 函数，传递参数为一个时间点 Date 对象，函数会查询药品处方记录表返回这个时间段内的药品处方记录列表。然后遍历药品处方记录表，判断处方项状态（2、未领药，3、已领药）进行分类，并且通过 2 个 Map 对象（Key 为患者挂号 ID，Value 为该挂号患者的处方记录 ID 列表记录），一个 Map 存储未领药患者的记录，另外一个 Map 存储已领药患者的记录。然后遍历 2 个 Map 根据 Key 即病人挂号 ID，调用挂号服务的 selectRegistrationById 方法得到挂号表信息，外加根据得到的挂号表信息中的病人 ID 调用用户服务的 selectPatientById 方法得到病人信息。最后对病人的信息进行封装返回封装的对象即可。

5.3 系统测试

软件测试是保证软件质量的重要方法，在系统开发完毕后，通过对系统进行测试来确定功能是否按预期的目标完成^[43,44]。本小节先介绍测试背景，然后分别对模块接口和功能设计测试用例，来检验系统的功能完成情况。最后需要考虑系统在并发环境下的承受能力，通过软件压测工具模拟多用户访问得到压测结果，分析在当前实验环境配置下的系统性能。

5.3.1 系统测试环境

系统测试环境应该尽可能的贴近生产环境，从而更好的模拟真实的运行情况，测试的结果也更具准确性。本系统的测试环境配置如下：

(1) 硬件环境

服务器：一台阿里云服务器和一台 64G 内存物理主机

客户端：笔记本电脑一台

(2) 软件环境

服务器操作系统：Centos 7.6

Java 版本：1.8.0

数据库版本：Mysql 5.7

应用服务器：Tomcat 9.0

Redis 版本：6.0

Nginx 版本：1.18.0

5.3.2 功能测试

功能测试主要是模拟用户对系统的使用，来检测系统的功能是否完善，是否满足预期的需求。下面将对系统的用户信息管理、预约挂号管理、门诊医生管理、收费管理 4 个模块分别进行功能测试。

对于用户管理模块，主要测试用户的登录、非法访问、传递错误的 Token 是否能成功、修改用户角色信息等内容。具体的测试用例，如表 5-1 所示。

表 5-1 用户管理测试用例

编号	用例名称	步骤	预期结果	实际结果
1-101	正确登录	输入正确用户名及密码，点击登录	页面跳转到后台首页	通过
1-102	非法访问	Postman 中直接访问不属于该角色的接口	提示权限不足	通过
1-103	非法 Token	Postman 中调用查看用户信息，传递的 Token 非法	返回 Token 非法	通过
1-104	修改用户角色	选中单个用户，点击修改按钮，修改其角色	提示修改成功	通过

挂号预约模块，主要测试用户是否能看到医生的坐诊信息、能否挂号成功及就医提醒是否能生效。具体的测试用例表如下表 5-2 所示。

表 5-2 挂号预约用例测试

编号	用例名称	步骤	预期结果	实际结果
2-101	查看坐诊信息	用户进入预约挂号界面，选中对应科室	展示科室下的医生坐诊信息	通过
2-102	挂号预约	点击可用的挂号按钮	提示挂号成功	通过
2-103	就医提醒	Postman, 调用短信发送服务	用户收到就医短信	通过

门诊医生管理模块，主要测试门诊医生是否能看到自己的患者列表、查看患者就诊信息、提交诊断病历、开立检查、开立药品；医技医生上传检查结果。具体的测试用例表如下表 5-3 所示。

表 5-3 门诊医生管理测试用例

编号	用例名称	步骤	预期结果	实际结果
3-101	查看患者就诊信息	点击左侧患者，在弹出提示中，点击确定诊断该患者按钮	以表格的形式展示患者的诊断信息	通过
3-102	提交诊断病历	填写患者就诊信息，点击提交按钮	提示保存成功	通过
3-103	添加检查项	点击添加项目按钮，选中对应项目，在弹出的提示框中选中确定添加按钮	检查申请列表表格中增加了该记录项	通过
3-104	开立项目	选中开立项目，点击开立项目按钮	提示开立成功，页表刷新显示项目从未开立转变为已开立	通过
3-105	上传检查结果	点击上传结果标签，输入检查结果，点击上传图片按钮，选中图片，最后点击提交。	提示上传结果成功	通过
3-106	添加药品	点击添加药品按钮，选中对应药品，在弹出的提示框中选中确定添加按钮	在处方页面显示添加的药品	通过
3-107	开立处方	勾选需要开立的处方，点击开立处方按钮	提示开立成功	通过

收费管理模块，主要测试展示患者的收费项目及收费，具体的测试用例表如下表 5-4 所示。

表 5-4 收费管理测试用例

编号	用例名称	步骤	预期结果	实际结果
4-101	展示患者收费列表	点击门诊收费工作台, 进入收费页面	页面展示患者的未/已收费信息	通过
4-102	收费	勾选需要收费的列表, 点击缴费按钮, 在弹出的确认收费框中输入应缴金额, 点击确认按钮	提示缴费成功	通过

5.3.2 非功能测试

性能测试主要测试的是系统在现有配置下能发挥的最大性能, 通过模拟不同用户数量向系统并发请求时的吞吐量、平均响应时间, 依据测试结果来分析系统的处理能力和负载能力^[45,46]。本文使用 Apache Benchmark 工具作为压力测试工具, 其能模拟不同数量的用户并发的对系统发起请求, 并统计出性能测试数据。

下面给出系统接口在不同的并发用户数据下的测试结果, 具体结果如下表 5-5 所示。

表 5-5 并发测试数据

并发数	请求数	吞吐量(RPS)	平均响应时间(ms)
10	100	890.31	1.132
10	500	989.32	0.961
20	100	1021.10	0.987
20	500	1052.16	0.940
100	100	1067.32	0.952
100	500	1116.09	0.961
100	10000	1043.87	0.931
1000	10000	1062.23	0.916

从上表数据中可以看到，在总请求数一样的情况下，系统并发数越大，系统的吞吐量越大，平均响应时间也越短，系统也能更好的发挥其性能。同时当并发数量一样的情况下，请求数的增大也会提升系统的吞吐量。比如当并发数为 20 时，发送 100 个和 500 个请求时，系统的吞吐量会有所增加。测试所得系统的平均吞吐量在 1026 Request/s，能够基本满足本系统的需求。

5.4 本章小结

本章首先介绍了构建本系统的开发框架及软件工具，然后详细介绍了医院门诊系统各个模块的实现过程，展示了各个模块的部分功能的代码实现及具体实现界面图。最后对系统进行了测试，从功能及性能两方面进行了详细的测试，对系统的接口及功能给出测试用例表；对系统的性能给出相应的并发用例数据，且基于这些数据对系统的性能进行分析总结。

6 总结与展望

6.1 全文总结

对于医院门诊信息管理系统的设计研发,本文先阐述了系统的相关背景及研究意义,接着对开发部署过程中遇到的关键技术进行基本介绍,为后序的开发打下了技术基础。接着对系统进行需求分析,充分理解用户的系统需求是系统设计的基石,只有深入的了解需求后,才能对系统的功能模块进行准确的划分及对角色该有的职责进行用例规划。其次对系统进行了分模块操作,给系统的设计明确了方向,最后系统被划分为用户信息管理、医生排班管理、预约挂号管理、门诊医生工作台、门诊收费管理、门诊药房管理六个模块。在介绍具体模块设计时,给出了每个功能模块的设计流程图,且对流程进行了相关阐述,将设计背后的思路充分体现了出来。系统的实现部分遵循设计模块的设计思路按期完成,且给出了具体的页面实现图及部分代码。最后对系统进行了相关测试,包括功能测试及非功能测试,确保每个模块的都能提供正常的功能。

在系统架构上,考虑到实际应用环境、开发成本及时间关系,最终决定采用B/S架构进行开发,采用前后端分离的方式,后端系统采用Java作为开发语言,使用SpringBoot + Mybatis作为搭建Web应用的基础,前端使用Vue框架进行组件化开发。考虑到系统在高并发场景下面临的压力,决定后端系统多部署,使用Nginx作为负载均衡工具,分流单系统面临的压力。最后考虑到业务需要,使用阿里云的OSS服务对本系统的图像进行存储,使用Kafka消息队列对延迟消息进行异步处理,并基于此消息队列开发就医短信提醒及处方过期功能。

整个系统开发过程严格遵循软件开发流程,医院门诊管理系统作为提高医院运维效率的管理工具,为患者、门诊医生等提供关键数据信息来源,有助于简化患者及医生的就诊过程。

6.2 展望

本系统虽然达到了预期的目标,完成了其中每个模块的功能,但是还存在许多可以提升的空间,需要后序的完善与优化,需要改进的部分如下。

本系统使用了 Nginx 的负载均衡机制分流了部分请求，解决了部分高并发的问題，但是后端整体还是单体应用，各个模块还是整合在一起，需要后序将每个模块都分离出来作为单独的一个模块进行部署。在日志方面，虽然使用了 logback 框架进行了日志收集，但是没有可视化的展示平台，通过进入文件查看日志实为不妥，需要使用 ELK（ElasticSearch+Logstash+Kibana）来搭建日志收集和可视化平台，这样通过页面即可直观的展示每个服务的运行状态，大大的简化了运维者的工作。

本系统虽然完成了各个模块基本且核心的功能，但是还有一些额外的功能需要添加，这样才能形成完整的业务链，真正实现商业化。医院门诊管理系统是一个庞大且要求严格的系统，如果需要更加完善添加更多定制功能，就需要投入更多的财力人力及物力。

致 谢

本论文在导师区士颀和薛志东教授的精心指导下完成,在论文项目的技术选型、功能模块设计及最终的修改阶段,导师抽出宝贵的时间,给与了许多诚恳的建议。导师博学的专业知识、严谨认真的治学态度、平易近人的待人方式、开拓进取的精神、严以律己的崇高风范使我受益匪浅,除了专业的知识外,更多地学到了待人接物的道理。

在华中科技大学我度过了充实的三年硕士生活,学到了许多专业知识,也结识了诸多朋友。感谢华中科技大学给我的这次读研深造机会,使我在学业知识上更上一层楼。在此也感谢软件学院的老师,他们生动且丰富的课程,大大的开阔了我的知识面,使我在学习的路上有了更明确的方向。同时感谢身边的同学及朋友,我们一起逐梦,一起努力,在学业和生活上互助互勉。

学业生涯即将结束,在此感谢我的家人,感谢他们一路以来给与我的关心、鼓舞、理解。

参考文献

- [1] 张宏斌. 中小型医院信息化建设中存在的问题和对策. 科技创新导报, 2018, 15(11): 175+177.
- [2] 陈熙文. 医院绩效管理信息系统的设计与实现. 世界最新医学信息文摘, 2019, 19(50): 257-258.
- [3] Ahmadi M, Aslani N. Capabilities and advantages of cloud computing in the implementation of electronic health record. *Acta Informatica Medica*, 2018, 26(1): 24.
- [4] 周晓叶. 云存储技术在医院信息管理中的应用. 河北农机, 2020(05): 124.
- [5] Smits M, Van Der Pijl G. Developments in hospital management and information systems//Proceedings of the 32nd Annual Hawaii International Conference on Systems Sciences. 1999. HICSS-32. Abstracts and CD-ROM of Full Papers. IEEE, 1999: 9 pp.
- [6] 俞颖婕. 中小型医院门诊管理信息系统的设计与实现:[硕士学位论文]. 北京: 北京交通大学, 2019
- [7] Kleinke J D. Dot-gov: market failure and the creation of a national health information technology system. *Health affairs*, 2005, 24(5): 1246-1262.
- [8] 凌红, 陈龙. 发达国家医院信息系统发展研究及启示. 中国医院管理, 2014, 34(06): 78-80.
- [9] 刘明. 基于 Web 的医院诊疗系统的设计与实现:[硕士学位论文]. 吉林: 吉林大学, 2018
- [10] 高扬. 基于 ASP.NET 的医院信息管理系统的设计与实现:[硕士学位论文]. 北京: 北京交通大学, 2020
- [11] 王晓天. 基于信息技术发展的医院信息化系统建设. 医学信息, 2019, 32(06): 25-26.
- [12] 刘鹏, 张辉. 医院信息化管理建设在医院发展中的作用. 信息记录材料, 2019, 20(03): 212-213.
- [13] 吴宝花. 基于 Nginx 的服务器集群负载均衡策略研究与优化:[硕士学位论文].

- [江西]: 南昌大学,2020
- [14] 刘双.Spring 框架中 IOC 的实现.电子技术与软件工程,2018(21):231.
- [15] Shen L, Liu Z, An P, et al. Low-Complexity Mode Decision for MVC. IEEE Transactions on Circuits&Systems for Video Technology, 2011, 21(6): 837-843
- [16] 罗凡, 裴士辉, 张雪松. MySQL 中 InnoDB 引擎的动态存储管理. 东北师大学报(自然科学版)2006(1): 22-26
- [17] James Berrington. Databases Anaesthesia & Intensive Care Medicine, 2013, 13(125): 78-80
- [18] Johnson R. Expert one-on-one J2EE design and development. John Wiley & Sons, 2004.
- [19] 陈小虎, 邓惠俊. 基于 mybatis 的数据持久层研究. 成都工业学院学报, 2020, 23(02): 29-32.
- [20] Shiyong X, Tianxiang D U, Rongzheng Z, et al. Research On Mybatis Mapper Model Based On SQL Template//2020 International Conference on Computer Engineering and Application (ICCEA). IEEE, 2020: 502-505.
- [21] 吕宇琛. SpringBoot 框架在 web 应用开发中的探讨. 科技创新导报, 2018, 15(08): 168+173.
- [22] Suryotrisongko H, Jayanto D P, Tjahyanto A. Design and development of backend application for public complaint systems using microservice spring boot. Procedia Computer Science, 2017, 124: 736-743.
- [23] Reddy K S P. Beginning Spring Boot 2: Applications and Microservices with the Spring Framework. Apress, 2017.
- [24] 邓笑. 基于 Spring Boot 的校园轻博客系统的设计与实现:[硕士学位论文]. 湖北: 华中科技大学, 2018
- [25] Yun-ping Y, Zhi-jun W U. Application of Apache Shiro Security Framework in Technology Transfer Services System. Computer and Modernization, 2014 (3): 158.
- [26] 梁清华, 胡安明. Apache Shiro 框架在 Web 系统的安全应用研究. 电脑知识与技术, 2021, 17(06): 52-53.
- [27] 傅安中, 王祥荣. 基于 Apache Shiro 的储物管理系统权限管理研究与实现. 电脑

- 编程技巧与维护,2020(03):99-102.
- [28] 许红军.使用安全机制管理 Kafka 消息队列.网络安全和信息化,2020(05):134-139.
- [29] Chai X C, Wang Q L, Chen W S, et al. Research on a distributed processing model based on Kafka for large-scale seismic waveform data. IEEE Access, 2020, 8: 39971-39981.
- [30] 孟亚辉.浅谈软件项目开发过程中的需求分析.科技信息,2009(11):435-436.
- [31] Jun F. A Brief Talk on Requirement Analysis of Software Project. Computer Development & Applications, 2012, 1.
- [32] 朱冬旭.计算机技术在医院信息管理系统中的应用.电子世界 2019(11):136-138.
- [33] 张翔翹.基于 WEB 技术的医院信息系统的设计与实现:[硕士学位论文]. 吉林: 吉林大学,2019
- [34] Chen L, Babar M A, Nuseibeh B. Characterizing architecturally significant requirements. IEEE software, 2012, 30(2): 38-45.
- [35] 李莉,张玉魁,龚勇浩.敏捷项目中管理非功能需求.项目管理技术,2020,18(08):85-88.
- [36] 李云云.浅析 B/S 和 C/S 体系结构.科学之友,2011(01):6-8.
- [37] Wang L. Study on B/S -based enterprise item bank management system, Green Communications and Networks, 2014, 54: 59.
- [38] 吴昌政.基于前后端分离技术的 web 开发框架设计:[硕士学位论文]. 南京: 南京邮电大学,2020
- [39] Hope R M, Schoelles M J, Gray W D. Simplifying the interaction between cognitive models and task environments with the JSON Network Interface. Behavior research methods, 2014, 46(4): 1007-1012.
- [40] 丘杰雄.Nginx+Keepalived+Tomcat+MySQL 高可用负载均衡 Web 应用架构实践.金融科技时代,2019(11):38-42.
- [41] 邱书洋.Redis 缓存技术研究及应用:[硕士学位论文]. 河南: 郑州大学,2016
- [42] Alshehri S, Raj R K. Secure access control for health information sharing systems//2013 IEEE international conference on healthcare informatics. IEEE,

2013: 277-286.

- [43] Pavani P N L, Prafulla B K, Rao R P, et al. Design, Modeling and Structural Analysis of Wave Springs. *Procedia materials science*, 2014, 6: 988-995.
- [44] 刘小龙. 基于需求的软件测试. *电子质量*, 2011, 15(7): 4-6
- [45] 沈涛. 综合电子信息系统软件质量的测试和评估. *计算机工程*, 2004, 7(3): 101-103
- [46] Tian J. *Software quality engineering: testing, quality assurance, and quantifiable improvement*. John Wiley & Sons, 2005.